Slick Barber Projekt Dokumentáció

Tartalomjegyzék

1. Fejlesztői dokumentáció 5
1.1. Bevezetés 5
1.1.1. Projekt rövid leírása 5
1.1.2. Fejlesztés során felhasznált technológiák 5
a. Szoftver technológiák 5
b. Konkrét szoftverek 6
1.1.3. Szerepkörök 6
a. Vendég / Guest 6
b. Borbély / Barber 6
c. Admin / Owner 6
1.1.4. Főbb funkciók 6
a. Vendégek számára 6
b. Barberek számára 7
c. Admin számára 7
d. Rendszer funkciók 7
e. Jogosultságkezelés 7
1.1.5. Célok és célkitűzések 7
a. Mit szeretnénk elérni a szoftverrel? 7
b. Rövid távú célok 7
c. Hosszú távú célok 7
1.1.6. Követelmények 8
a. Funkcionális követelmények 8
b. Nem-funkcionális követelmények 8
1.1.7. Alkalmazási területek 8
1.1.8. Fejlesztési módszertan 8
1.1.9. Csapat tagjainak szerepkörei 8
1.1.10. Ütemterv 9
1.2. Adatbázis dokumentáció 9
1.2.1. Bevezetés 9
1.2.2. Általános információk 9
1.2.3. Adatbázis szerkezet 10

a. Adatbázis diagram 10
b. Vendég tábla 11
c. Borbély tábla 12
d. Szolgáltatások tábla 13
e. Időpontok tábla 14
f. Munkaidő tábla 15
g. Log tábla 16
1.2.4. Adatbiztonság 16
1.2.5. Jövőbeni bővítések 17
1.3. Funkció és API dokumentáció 17
1.3.1. Barber felhasználókezelő API 17
a. Általános információk 17
b. Hibakódok 17
c. Végpontok 18
1.3.2. Vendégkezelő API 23
a. Általános információk 23
b. Hibakódok 23
c. Végpontok 24
1.3.3. Szolgáltatáskezelő API 29
a. Általános információk 29
b. Hibakódok 29
c. Végpontok 30
1.3.4. Időpontkezelő API 35
a. Általános információk 35
b. Hibakódok 35
c. Végpontok 36
1.3.5. Munkaidő / Nyitvatartás kezelő API 45
a. Általános információk 45
b. Hibakódok 45
c. Végpontok 46
1.3.6. Middleware logikája 52
1.4. Reszponzív weboldal 52
1.4.1. Technológiai architektúra: 52

1.4.2. Frontend komponensek és stílus: 53
1.4.3. Dinamikus modulok logikája: 54
1.4.4. Foglalási folyamat technikai leírása: 56
1.4.5. Külső szolgáltatások integrációja: 57
1.5. Fodrászszalon Kliens Alkalmazás 57
1.5.1. Specifikáció 57
1.5.2. Funkcionális követelmények 58
1.5.3. Nem-funkcionális követelmények 58
1.5.4. Használati Eset Diagram 59
1.5.5. Felhasználói történetek 59
1.5.6. Szoftver Architektúra és Tervezés 72
a. MVVM Architektúra 72
b. REST API Kommunikáció 73
c. ViewModels Osztálydiagram 74
d. Models Osztálydiagram 76
e. Services Osztálydiagram 77
f. View és ViewModel Osztálydiagram 78
g. Multimédia kezelés és Adatkonverzió 78
1.6. Tesztelési dokumentáció 81
1.6.1. Bevezetés 81
1.6.2. REST API Tesztelés 81
1.6.2.1. Tesztelési cél 81
1.6.2.2 Tesztelt modulok 81
1.6.2.3. Tesztkörnyezet 82
1.6.2.4 Automata tesztesetek 83
1.6.2.5. Hibák nyomon követése 89
1.6.2.6. Teszt összegzés 91
1.6.3. Reszponzív weboldal tesztelés 91
1.6.3.1. Tesztelési cél 91
1.6.2.2. Tesztelt modulok 91
1.6.2.3. Tesztkörnyezet 91
1.6.2.4. Teszt összegzés 92
1.6.4. Szoftvertesztelési dokumentáció 92

1.6.4.1. Manuális tesztelési dokumentáció 92
a. 1.6.4.1.1. Cél meghatározása 92
b. 1.6.4.1.2. Teszterv 92
c. 1.6.4.1.3. Tesztesetek 92
1.6.4.2. Automata tesztelési dokumentáció 94
1.6.4.2.1. Cél meghatározása 94
1.6.4.2.2. Teszterv 94
1.6.4.2.3. Automata tesztesetek 94
1.6.4.3. Teszt összegzés 94
2. Felhasználói dokumentáció 95
2.1. Bevezetés 95
2.2. Vendégek számára (Webes felület) 95
2.2.1. Navigáció 95
2.2.2. Regisztráció és bejelentkezés 96
2.2.3. Időpontfoglalás menete 97
2.2.4. Profil kezelése és Foglalások lemondása 99
2.3. Borbélyok számára (Asztali és Mobil alkalmazás) 101
2.3.1. Telepítés és futtatás 101
2.3.2. Belépés 103
2.3.3. Naptár és Időpontok kezelése 104
2.3.4. Szolgáltatások és Munkaidő beállítása 106
2.3.5. Profil és kijelentkezés 108
2.4. Adminisztrátorok számára (Üzletvezetők) 109
2.4.1. Felhasználók kezelése (Admin Panel) 109
2.4.2. Rendszernapló (Logs) 110

1. Fejlesztői dokumentáció

1.1. Bevezetés

1.1.1. Projekt rövid leírása

A projekt célja egy komplex fodrászat kezelő platform fejlesztése, amely két fő részből áll: egy kliensoldali webes felületből a vendégek számára, ahol böngészhetik a szolgáltatásokat és időpontot foglalhatnak, valamint egy keresztplatformos (asztali és mobil) menedzser alkalmazásból a fodrászok számára, amellyel kezelhetik naptárukat, szolgáltatásaikat és munkaidejüket. A rendszer automatikus email-visszaigazolással és naplózással segíti a szalon napi működését.

1.1.2. Fejlesztés során felhasznált technológiák

a. Szoftver technológiák

- **Frontend:** HTML5 – a weboldalak alapvető leíró nyelve, amely a tartalom szerkezetét határozza meg elemek és tagek segítségével.
- **CSS3** – stílusleíró nyelv, amely a weboldalak megjelenését szabályozza (színek, betűtípusok, elrendezés, reszponzív design), és a HTML struktúráját vizuálisan formázza.
- **(Bootstrap 5)** – frontend CSS keretrendszer, amely előre elkészített komponenseket és reszponzív rácsrendszert biztosít gyors és egységes megjelenésű weboldalak készítéséhez.
- **Backend:** Node.js - szerver oldali JavaScript futtatókörnyezet, amely lehetővé teszi backend alkalmazások és API-k fejlesztését eseményvezérelt, nem blokkoló működéssel.
- **Adatkezelés:** Sequelize ORM - MySQL adatbázis-kezelővel a strukturált adattároláshoz

- **Menedzser App (Fodrász):** C# .NET 8 és Avalonia UI - MVVM amely asztali és mobil környezetben is fut
- **Autentikáció:** JWT (JSON Web Tokens) és Bcrypt jelszó titkosítás

b. Konkrét szoftverek

- **IDE:** Visual Studio Code (Backend/Web), Visual Studio 2022 (C# fejlesztés).
- **Verziókezelés:** GitHub – kódtár és verziókövetés
- **Tesztelés:** Jest és Supertest (Backend API tesztek), Postman (API végpontok manuális tesztelése).
- **Adatbázis-kezelés:** MySQL Workbench vagy phpMyAdmin.

1.1.3. Szerepkörök

a. Vendég / Guest

- **Funkciók:** Regisztráció, bejelentkezés, borbélyok és szolgáltatások megtekintése, szabad időpontok keresése, foglalás leadása, saját foglalások kezelése, fiókjának módosítása illetve törlése.

b. Borbély / Barber

- **Funkciók:** Szolgáltatások felvétele/módosítása/törlése, munkaidő beállítása napokra lebontva, naptárkezelés (foglalások megtekintése, késznek jelölése vagy törlése), rendszernapló ellenőrzése.

c. Admin / Owner

- **Funkciók:** Profilok szerkesztése, felvétele/módosítása/törlése

1.1.4. Főbb funkciók

a. Vendégek számára

- Interaktív naptár alapú időpontválasztás.

- Automatikus email visszaigazolás a sikeres foglalásról.
- Saját foglalások nyomon követése és lemondása a webes felületen.

b. Barberek számára

- Dinamikus heti naptár nézet az asztali alkalmazásban.
- Munkaórák precíz beállítása
- Szolgáltatások teljes körű menedzselése.

c. Admin számára

- Felhasználók teljes körű kezelése
- Logok megtekintése

d. Rendszer funkciók

- **Naplózás (Logging)**: Minden fontosabb művelet (foglalás módosítása, törlése) rögzítésre kerül a rendszernaplóba.
- **Ütközésvédelem**: A rendszer nem enged két foglalást ugyanarra az időpontra ugyanahhoz a fodrászhoz.

e. Jogosultságkezelés

- A fodrászok csak a saját szolgáltatásaikat és naptárukat szerkeszthetik.
- A vendégek csak a saját profiljához tartozó foglalásokat láthatják/módosíthatják.

1.1.5. Célok és célkitűzések

a. Mit szeretnénk elérni a szoftverrel?

- Egy modern, papírmentes és automatizált ügyfélkezelési folyamat kialakítása, amely minimalizálja az adminisztrációs terheket a fodrászok számára.

b. Rövid távú célok

- A stabil foglalási rendszer és a keresztplatformos fodrász alkalmazás implementálása alapvető CRUD funkciókkal

c. Hosszú távú célok

- Online fizetési rendszer integrálása, statisztikák és bevételi kimutatások készítése a borbélyok számára, push-értesítések mobilra.

1.1.6. Követelmények

a. Funkcionális követelmények

- Biztonságos regisztráció és bejelentkezés.
- Valós idejű szabad időpont keresés a fodrász munkaideje és meglévő foglalásai alapján.
- Sikeres foglalás utáni azonnali email értesítés.

b. Nem-funkcionális követelmények

- **Válaszidő**: Az API lekérdezéseknek 200ms alatt kell maradniuk.
- **Elérhetőség**: A menedzser alkalmazásnak asztali gépen és Androidos mobilon is azonos funkcionalitást kell biztosítania.
- **Adatvédelem**: A felhasználói jelszavak csak titkosított formában tárolhatóak.

1.1.7. Alkalmazási területek

A szoftver elsősorban kisebb-közepes fodrász szalonok és magán fodrászok számára készült, de adaptálható bármilyen időpont-alapú szolgáltatást nyújtó vállalkozás számára (pl. kozmetika, masszázs).

1.1.8. Fejlesztési módszertan

Agilis fejlesztési modellt alkalmazunk, 2 hetes sprintekkel. Minden sprint végén teszteljük az új funkciókat a backend és a frontend/menedzser app között.

- Folyamatos integráció a backend API és a C# kliens között.
- Egységtesztelés Jest keretrendszerrel a kritikus üzleti logikákra (pl. slot generálás).

1.1.9. Csapat tagjainak szerepkörei

- **Frontend:** Gózon Szabolcs Dániel
- **Backend:** Patai Krisztián
- **Adatbázis:** Patai Krisztián

- **Asztali & mobil alkalmazás:** Ujvári Csongor
- **Tesztelés:**
    - **Manuális tesztelés**: Gózon Szabolcs Dániel, Patai Krisztián, Ujvári Csongor
    - **Automata (unit) tesztelés**: Patai Krisztián, Ujvári Csongor
- **Dokumentáció készítése és frissítése:** Patai Krisztián, Gózon Szabolcs Dániel, Ujvári Csongor

1.1.10. Ütemterv

1. **Kezdeti tervezés (1. hónap):**
    - Adatbázis séma véglegesítése, API végpontok specifikálása.
    - Frontend kinézetének megtervezése
2. **Első prototípus (2-3. hónap):**
    - Autentikáció, alapvető CRUD műveletek (vendég/fodrász/szolgáltatások).
3. **Béta verzió (4-5. hónap):**
    - A webes foglalási felület és az Avalonia alapú asztali alkalmazás párhuzamos fejlesztése.
4. **Élesítés (6. hónap):**
    - Élesítés, dokumentáció lezárása

1.2. Adatbázis dokumentáció

1.2.1. Bevezetés

Ez a dokumentáció a felhasználói regisztrációhoz használt adatbázis szerkezetét és működését ismerteti. Az adatbázis célja a felhasználók adatainak biztonságos tárolása és kezelése.

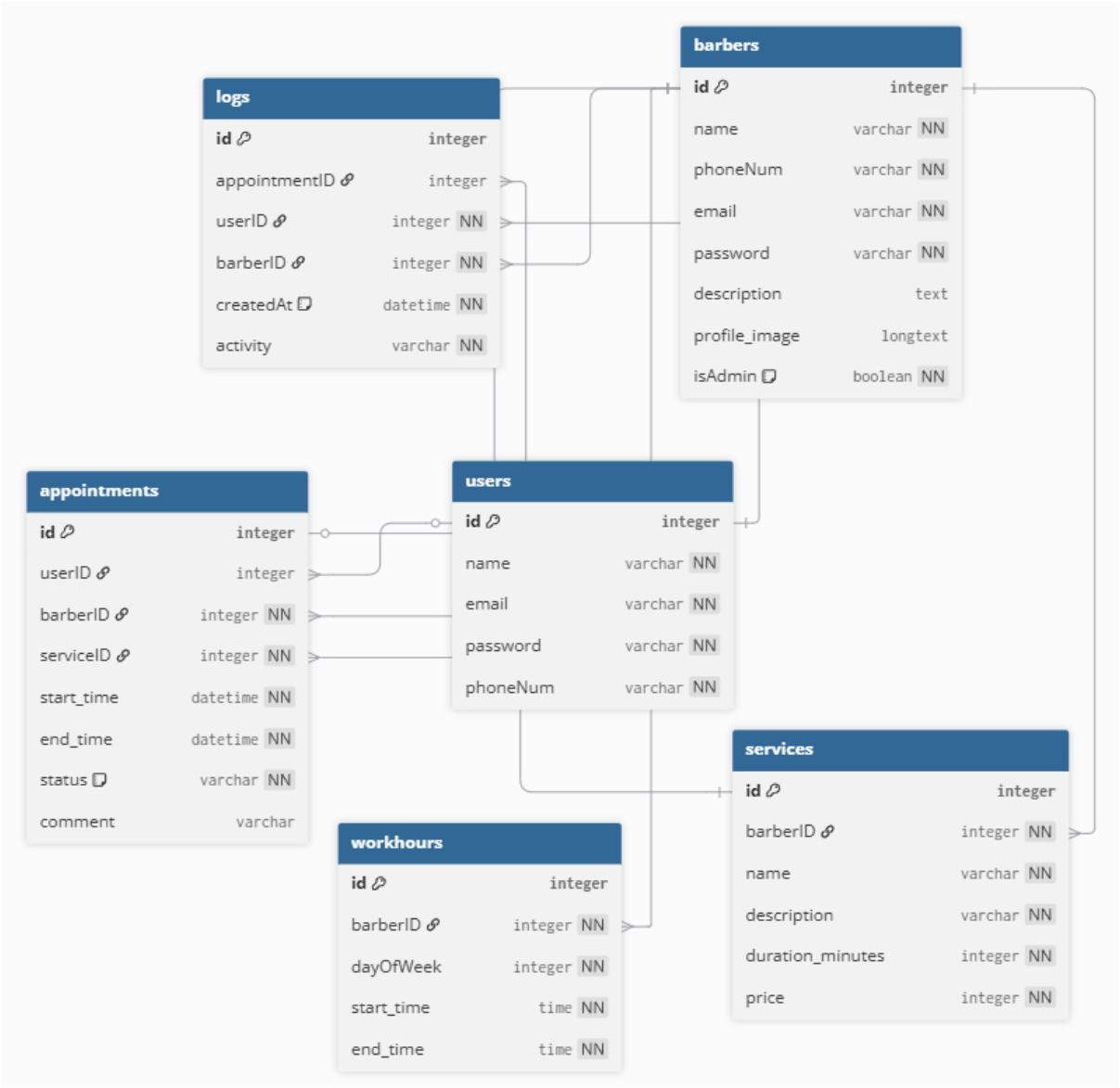
1.2.2. Általános információk

- **Adatbázis neve**: barbershop
- **Adatbázis-kezelő rendszer (DBMS)**: MySQL
- **Verzió**: 8.0

- **Készítők**: Patai Krisztián
- **Dátum**: 2025. február 12.

1.2.3. Adatbázis szerkezet

a. Adatbázis diagram



b. Vendég tábla

Leírás: A vendégekhez rendelt adatokat tárolja.

Mező neve	Típusa	Leírás	Korlátozások
id	INT(11)	Egyedi azonosító	PRIMARY KEY, AUTO_INCREMENT
name	VARCHAR(255)	vendég neve	NOT NULL
email	VARCHAR(255)	vendég E-mail címe	NOT NULL
password	VARCHAR(255)	Titkosított jelszó (bcrypt)	NOT NULL
phoneNum	VARCHAR(255)	vendég telefonszáma	NOT NULL

Példa a tábla létrehozására:

```
CREATE TABLE users(
    id = INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    email VARCHAR(255) NOT NULL,
    password VARCHAR(255) NOT NULL,
    phoneNum VARCHAR(255) NOT NULL
)
```

c. Borbély tábla

Leírás: A borbélyokhoz rendelt adatokat tárolja.

Mező neve	Típusa	Leírás	Korlátozások
id	INT(11)	Egyedi azonosító	PRIMARY KEY, AUTO_INCREMENT
name	VARCHAR(255)	borbély neve	NOT NULL
phoneNum	VARCHAR(255)	borbély telefonszáma	NOT NULL
email	VARCHAR(255)	borbélyE-mail címe	NOT NULL
password	VARCHAR(255)	Titkosított jelszó (bcrypt)	NOT NULL
isAdmin	BOOLEAN	Admin-e	DEFAULT FALSE

Példa a tábla létrehozására:

```
CREATE TABLE barbers (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    phoneNum INT NOT NULL,
    email VARCHAR(255) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    isAdmin BOOLEAN NOT NULL DEFAULT false,
  )
```

d. Szolgáltatások tábla

Leírás: A szolgáltatásokhoz rendelt adatokat tárolja.

Mező neve	Típusa	Leírás	Korlátozások
id	INT(11)	Egyedi azonosító	PRIMARY KEY, AUTO_INCREMENT
BarberID	INT(11)	borbély ID-ja	NOT NULL
name	VARCHAR(255)	szolgáltatás neve	NOT NULL
description	VARCHAR(255)	szolgáltatás leírása	NOT NULL
duration_minutes	INT(11)	szolgáltatás hossza (percben)	NOT NULL
price	INT(11)	szolgáltatás ára	NOT NULL

Példa a tábla létrehozására:

```
CREATE TABLE services (
    id INT AUTO_INCREMENT PRIMARY KEY,
    barberID INT NOT NULL,
    name VARCHAR(255) NOT NULL,
    description VARCHAR(255) NOT NULL,
    duration_minutes INT NOT NULL,
    price INT NOT NULL,
    FOREIGN KEY (barberID) REFERENCES barbers(id) ON DELETE CASCADE
);
```

e. Időpontok tábla

Leírás: Az időpontokhoz rendelt adatokat tárolja.

Mező neve	Típusa	Leírás	Korlátozások
id	INT(11)	Egyedi azonosító	PRIMARY KEY, AUTO_INCREMENT
userID	INT(11)	vendég ID-ja	NOT NULL
BarberID	INT(11)	borbély ID-ja	NOT NULL
ServiceID	INT(11)	szolgáltatás ID-ja	NOT NULL
start_time	DATETIME	időpont kezdete	NOT NULL
end_time	DATETIME	időpont vége	NOT NULL
status	VARCHAR(255)	időpont státusza foglalt-e vagy nem	NOT NULL
comment	VARCHAR	időponthoz megjegyzés	NOT NULL

Példa a tábla létrehozására:

```sql
CREATE TABLE appointments (
    id INT AUTO_INCREMENT PRIMARY KEY,
    userID INT NULL,
    barberID INT NOT NULL,
    serviceID INT NOT NULL,
    start_time DATETIME NOT NULL,
    end_time DATETIME NOT NULL,
    status VARCHAR(255) NOT NULL DEFAULT 'booked',
    comment VARCHAR(255) NULL,
    FOREIGN KEY (userID) REFERENCES users(id) ON DELETE SET NULL,
    FOREIGN KEY (barberID) REFERENCES barbers(id) ON DELETE CASCADE,
    FOREIGN KEY (serviceID) REFERENCES services(id) ON DELETE CASCADE
);
```

f. Munkaidő tábla

Leírás: A beosztásokhoz rendelt adatokat tárolja.

Mező neve	Típusa	Leírás	Korlátozások
id	INT(11)	Egyedi azonosító	PRIMARY KEY, AUTO_INCREMENT
BarberID	INT(11)	borbély ID-ja	NOT NULL
dayOfWeek	INT(11)	szolgáltatás ID-ja	NOT NULL
start_time	TIME	Borbély munkaidejének kezdete	NOT NULL
end_time	TIME	Borbély munkaidejének vége	NOT NULL

Példa a tábla létrehozására:

```
CREATE TABLE workhours (
    id INT AUTO_INCREMENT PRIMARY KEY,
    barberID INT NOT NULL,
    dayOfWeek INT NOT NULL,
    start_time TIME NOT NULL,
    end_time TIME NOT NULL,
    FOREIGN KEY (barberID) REFERENCES barbers(id) ON DELETE CASCADE
);
```

g. Log tábla

Leírás: Logolja az időpont lemondásokat és törléseket.

Mező neve	Típusa	Leírás	Korlátozások
id	INT(11)	Egyedi azonosító	PRIMARY KEY, AUTO_INCREMENT
userID	INT(11)	vendég ID-ja	NOT NULL
barberID	INT(11)	borbély ID-ja	NOT NULL
appointmentID	INT(11)	időpont ID-ja	NOT NULL
activity	VARCHAR(255)	Az időpont státusza	NOT NULL

Példa a tábla létrehozására:

```
CREATE TABLE logs (
    id INT AUTO_INCREMENT PRIMARY KEY,
    appointmentID INT NULL,
    userID INT NOT NULL,
    barberID INT NOT NULL,
    activity VARCHAR(255) NOT NULL,

    FOREIGN KEY (appointmentID) REFERENCES appointments(id) ON DELETE SET NULL,
    FOREIGN KEY (userID) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (barberID) REFERENCES barbers(id) ON DELETE CASCADE
);
```

1.2.4. Adatbiztonság

- **Jelszó hashelés**: A jelszavakat bcrypt algoritmussal hasheljük, amely biztonságos és széles körben elfogadott módszer a jelszavak tárolására.
- **Adatvédelem**: Az e-mail címek és felhasználónevek egyedi korlátozása megakadályozza a duplikált adatok tárolását, csökkentve ezzel az adatbázis inkonzisztenciájának kockázatát.

1.2.5. Jövőbeni bővítések

- **Dinamikus idősáv-kezelés:** A szabad időpontok intelligens kalkulálása a borbély munkaideje és a kiválasztott szolgáltatás egyedi időtartama alapján, biztosítva a naptár optimális, hézagmentes kihasználtságát.
- **Kétfaktoros hitelesítés (2FA)**: Extra védelmi réteg a borbélyok és adminisztrátorok belépéséhez.
- **Értékelési rendszer**: A vendégek véleményezhetik és csillagokkal értékelhetik a borbélyokat a látogatás után, segítve ezzel az új ügyfelek döntését.

1.3. Funkció és API dokumentáció

1.3.1. Barber felhasználókezelő API

Az API célja, hogy biztosítsa a borbély (felhasználó) adatok kezelését az asztali alkalmazás és backend között. Lehetővé teszi a felhasználók regisztrációját, bejelentkezését, adatainak lekérését, módosítását és törlését.

a. Általános információk

- **Bázis URL**: https://api.slickbarber.hu
- **Protokoll**: HTTPS
- **Formátum**: JSON
- **Autentikáció**: Bearer Token (JWT)

b. Hibakódok

Hibakód	Leírás
200/201	Sikeres kérés
400	Helytelen kérés
401	Jogosultság hiánya
404	Nem található
500	Szerverhiba

c. Végpontok

GET /barberGet

Leírás: Az összes borbély adatainak lekérdezése.

Várható bemenetek: **Header**: Authorization: Bearer {token}

Válasz formátuma:

```
{
  "id": 1,
  "name": "Barber",
  "email": "Barber@example.com"
},
{
  "id": 2,
  "name": "Jane Smith",
  "email": "janesmith@example.com"
}
```

Hibakezelés: 401: "Unauthorized" - Hibás vagy hiányzó token.

POST /barberReg

Leírás: Borbély regisztrációja.

Várható bemenetek:

- **Body**:

```
{
    "name": "Barber",
    "email": "Barber@example.com",
    "password": "BarberPassword",
    "phoneNum": 12345
}
```

Válasz formátuma:

```
{
    "Sikeres regisztráció"
    "id": 3
}
```

1. **Hibakezelés**: 400: "missing data" - Hiányos paraméterek.
2. **Hibakezelés:** 400 “Már létező barber” - Már létező E-mail cím

POST /barberLogin

Leírás: Borbély bejelenkezése.

- **Body**:

```
{
    "name": "Barber",
    "email": "Barber@example.com",
    "password": "BarberPassword"
}
```

Válasz formátuma:

```
{
    "Sikeres Bejelentkezés"
    "id": 3
}
```

1. **Hibakezelés**: 400: "missing data" - Hiányos paraméterek.
2. **Hibakezelés:** 400 “Hibás név/jelszó/E-mail cím” - hibás paraméterek
3. **Hibakezelés**: 500 “Szerverhiba” - Szerverhiba

PUT /barberUpdate

Leírás: Borbély adatainak módosítása.

Várható bemenetek:

- **Header**:
  Authorization: Bearer {token}
- **Body**

```
{
    "name": "Geri",
    "email": "Geri@example.com",
    "password": "GeriPassword",
    "phoneNum": 54321,
    "description":"Több éves tapasztalattal rendelkező",
    "profile_image":"..."
}
```

Válasz formátuma:

```
{
    "Sikeres Módosítás"
    "id": 3
}
```

1. **Hibakezelés**: 400: "Nincs módosítandó adat" - Hiányos paraméterek.
2. **Hibakezelés:** 400 “Invalid ID” - Rossz vagy nem létező ID
3. **Hibakezelés:** 401 “No token” - Nem létező vagy lejárt token
4. **Hibakezelés:** 404 “Nincs ilyen felhasználó - Nem létező felhasználó
5. **Hibakezelés**: 500 “Szerverhiba” - Szerverhiba

Delete /barberDelete

Leírás: Borbély törlése.

- **Header**:
  Authorization: Bearer {token}
- **Body**

Válasz formátuma:

```
{
    "Sikeres Törlés"
    "id": 3
}
```

1. **Hibakezelés:** 400 “Invalid ID” - Rossz vagy nem létező ID
2. **Hibakezelés:** 401 “No token” - Nem létező vagy lejárt token
3. **Hibakezelés:** 404 “Nincs ilyen felhasználó - Nem létező felhasználó
4. **Hibakezelés**: 500 “Szerverhiba” - Szerverhiba

1.3.2. Vendégkezelő API

Az API célja a felhasználók adatainak kezelése és a frontend–backend közötti kommunikáció biztosítása.

Lehetővé teszi a felhasználók regisztrációját, bejelentkezését, adatainak lekérését, módosítását és törlését

a. Általános információk

- **Bázis URL**: https://api.slickbarber.hu
- **Protokoll**: HTTPS
- **Formátum**: JSON
- **Autentikáció**: Bearer Token (JWT)

b. Hibakódok

Hibakód	Leírás
200/201	Sikeres kérés
400	Helytelen kérés
401	Jogosultság hiánya
404	Nem található
500	Szerverhiba

c. Végpontok

GET /userGet

Leírás: Az összes vendég adatainak lekérdezése.

Várható bemenetek: **Header**: Authorization: Bearer {token}

Válasz formátuma:

```
{
  "id": 1,
  "name": "user",
  "email": "user@example.com"
},
{
  "id": 2,
  "name": "user2",
  "email": "user2@example.com"
}
```

Hibakezelés: 401: "Unauthorized" - Hibás vagy hiányzó token.

POST /userReg

Leírás: Vendég regisztrációja.

Várható bemenetek:

- **Body**:

```
{
    "name": "user",
    "email": "user@example.com",
    "password": "userPassword",
    "phoneNum": 12345
}
```

Válasz formátuma:

```
{
    "Sikeres regisztráció"
    "id": 3
}
```

1. **Hibakezelés**: 400: "missing data" - Hiányos paraméterek.
2. **Hibakezelés:** 400 “Már létező User” - Már létező E-mail cím

POST /userLogin

Leírás: Vendég bejelenkezése.

- **Body**:

```
{
    "name": "user",
    "email": "user@example.com",
    "password": "userPassword"
}
```

Válasz formátuma:

```
{
    "Sikeres Bejelentkezés"
    "id": 3
}
```

1. **Hibakezelés**: 400: "missing data" - Hiányos paraméterek.
2. **Hibakezelés:** 400 “Hibás név/jelszó/E-mail cím” - hibás paraméterek
3. **Hibakezelés**: 500 “Szerverhiba” - Szerverhiba

PUT/userUpdate

Leírás: Vendégadatainak módosítása.

Várható bemenetek:

- **Header**:
  Authorization: Bearer {token}
- **Body**

```
{
    "name": "Dani",
    "email": "Dani@example.com",
    "password": "DaniPassword",
    "phoneNum": 54321
}
```

Válasz formátuma:

```
{
    "Sikeres Módosítás"
    "id": 3
}
```

1. **Hibakezelés**: 400: "Nincs módosítandó adat" - Hiányos paraméterek.
2. **Hibakezelés:** 400 “Invalid ID” - Rossz vagy nem létező ID
3. **Hibakezelés:** 401 “No token” - Nem létező vagy lejárt token
4. **Hibakezelés:** 404 “Nincs ilyen felhasználó - Nem létező felhasználó
5. **Hibakezelés**: 500 “Szerverhiba” - Szerverhiba

Delete/userDelete

Leírás: Vendég törlése.

Várható bemenetek:

- **Header**:
  Authorization: Bearer {token}
- **Body**

Válasz formátuma:

```
{
    "Sikeres Törlés"
    "id": 3
}
```

1. **Hibakezelés:** 400 “Invalid ID” - Rossz vagy nem létező ID
2. **Hibakezelés:** 401 “No token” - Nem létező vagy lejárt token
3. **Hibakezelés:** 404 “Nincs ilyen felhasználó - Nem létező felhasználó
4. **Hibakezelés**: 500 “Szerverhiba” - Szerverhiba

1.3.3. Szolgáltatáskezelő API

Az API célja a szolgáltatások kezelésének biztosítása a rendszerben.

Lehetővé teszi a szolgáltatások lekérését, létrehozását, módosítását és törlését.

A szolgáltatások egy adott borbélyhoz tartoznak, és csak a tulajdonos módosíthatja őket.

a. Általános információk

- **Bázis URL**: https://api.slickbarber.hu
- **Protokoll**: HTTPS
- **Formátum**: JSON
- **Autentikáció**: Bearer Token (JWT)

b. Hibakódok

Hibakód	Leírás
200/201	Sikeres kérés
400	Helytelen kérés
401	Jogosultság hiánya
404	Nem található
500	Szerverhiba

c. Végpontok

GET /servicesGet

Leírás: Az összes szolgáltatás adatainak lekérdezése.

Várható bemenetek:

- **Header**:
  Authorization: Bearer {token}

Válasz formátuma:

```
{
  "id": 1,
  "barberID": 1,
  "name": "vágás",
  "description": "leírás",
  "duration_minutes": 10,
  "price": 5000
},
{
  "id": 2,
  "barberID": 2,
  "name": "Festés",
  "description": "leírás",
  "duration_minutes": 60,
  "price": 9000
}
```

Hibakezelés: 401: "Unauthorized" - Hibás vagy hiányzó token.

GET /servicesMy

Leírás: A borbély saját szolgáltatásainak lekérdezése.

Várható bemenetek:

- **Header**:
  Authorization: Bearer {token}

Válasz formátuma:

```
{
  "id": 1,
  "barberID": 1,
  "name": "vágás",
  "description": "leírás",
  "duration_minutes": 10,
  "price": 5000
},
{
  "id": 3,
  "barberID": 1,
  "name": "Haj szakáll",
  "description": "leírás",
  "duration_minutes": 40,
  "price": 7000
}
```

Hibakezelés: 401: "Unauthorized" - Hibás vagy hiányzó token.

POST /servicesPost

Leírás:Szolgáltatás létrehozása.

Várható bemenetek:

- **Header**:

  Authorization: Bearer {token}

- **Body**

```
{
    "name": "hajvágás",
    "description": "leírás",
    "duration_minutes": 20,
    "price": 5000
}
```

Válasz formátuma:

```
{
    "Sikeres létrehozás"
    "id": 4
}
```

1. **Hibakezelés**: 400: "missing data" - Hiányos paraméterek.
2. **Hibakezelés:** 400 “Már létező User” - Már létező E-mail cím
3. **Hibakezelés**: 500 “Szerverhiba” - Szerverhiba

PUT/servicesUpdate

Leírás: Szolgáltatások adatainak módosítása.

Várható bemenetek:

- **Header**:
  Authorization: Bearer {token}
- **Body**

```
{
    "name": "hajvágáááás",
    "description": "nagyonkomoly"
    "duration_minutes": 15,
    "price": 6500
}
```

Válasz formátuma:

```
{
    "Sikeres Módosítás"
    "id": 3
}
```

1. **Hibakezelés**: 400: "Nincs módosítandó adat" - Hiányos paraméterek.
2. **Hibakezelés:** 400 “Invalid ID” - Rossz vagy nem létező ID
3. **Hibakezelés:** 401 “No token” - Nem létező vagy lejárt token
4. **Hibakezelés:** 404 “Nincs ilyen szolgáltatás- Nem létező szolgáltatás
5. **Hibakezelés**: 500 “Szerverhiba” - Szerverhiba

Hibakezelés: 500 “Szerverhiba” - Szerverhiba

Delete/servicesDelete

Leírás: Szolgáltatás törlése.

Várható bemenetek:

- **Header**:
  Authorization: Bearer {token}
- **Body**

Válasz formátuma:

```
{
    "Sikeres Törlés"
    "id": 3
}
```

1. **Hibakezelés:** 400 “Invalid ID” - Rossz vagy nem létező ID
2. **Hibakezelés:** 401 “No token” - Nem létező vagy lejárt token
3. **Hibakezelés:** 404 “Nincs ilyen felhasználó - Nem létező felhasználó
4. **Hibakezelés**: 500 “Szerverhiba” - Szerverhiba

1.3.4. Időpontkezelő API

Az API célja az időpontfoglalások kezelése a rendszerben. Lehetővé teszi a felhasználók és borbélyok számára az időpontok lekérését, foglalását, módosítását és törlését.

Az API biztosítja az ütközések kezelését, valamint a szabad időpontok lekérdezését is.

a. Általános információk

- **Bázis URL**: https://api.slickbarber.hu
- **Protokoll**: HTTPS
- **Formátum**: JSON
- **Autentikáció**: Bearer Token (JWT)

b. Hibakódok

Hibakód	Leírás
200/201	Sikeres kérés
400	Helytelen kérés
401	Jogosultság hiánya
404	Nem található
500	Szerverhiba

c. Végpontok

GET /appointmentMyBarber

Leírás: A borbély saját időpontjainak lekérdezése

Várható bemenetek:

- **Header**:
  Authorization: Bearer {token}

Válasz formátuma:

```
{
    "id": 1,
    "userID": 1,
    "barberID": 1,
    "serviceID": 1,
    "start_time": "2026-03-08 18:11:00",
    "end_time": "2026-03-08 18:16:00",
    "status": "booked",
    "comment": "megjegyzés"
  },


  "user":{
    "name": "username",
    "email": "username@example.com",
    "phoneNum": 12345
```

```
  },
  "service":{
  "name": "username",
  "price": 5000
 }
```

Hibakezelés: 401: "Unauthorized" - Hibás vagy hiányzó token.

GET /appointmentMyUser

Leírás: A vendég saját időpontjainak lekérdezése

Várható bemenetek:

- **Header**:
  Authorization: Bearer {token}

Válasz formátuma:

```
  {
    "id": 1,
    "userID": 1,
    "barberID": 1,
    "serviceID": 1,
    "start_time": "2026-03-08 18:00:00",
    "end_time": "2026-03-08 18:00:00",
    "status": "booked",
    "comment": "megjegyzés"
  },
```

```
  "barber":{
    "name": "Geri"
  },
  "service":{
  "name": "vagas"
 }
```

Hibakezelés: 401: "Unauthorized" - Hibás vagy hiányzó token.

GET /availableSlots

Leírás:A szabad időpontok lekérdezése

Várható bemenetek:

- **Header**:
  Authorization: Bearer {token}

Válasz formátuma:

```
[
    {
      "2026-03-25T09:00:00.000Z",
      "2026-03-25T09:30:00.000Z",
      "2026-03-25T10:00:00.000Z",
      "2026-03-25T10:30:00.000Z",
      "2026-03-25T11:00:00.000Z",
      "2026-03-25T11:30:00.000Z"
    }
]
```

Hibakezelés: 401: "Unauthorized" - Hibás vagy hiányzó token.

POST /appointmentPost

Leírás:Időpont létrehozása.

Várható bemenetek:

- **Header**:

  Authorization: Bearer {token}

- **Body**

```
  {
    "id": 4,
    "serviceID": 4,
    "userID": 4,
    "start_time": "2026-03-08 12:00:00",
    "end_time": "2026-03-08 12:30:00",
    "comment": "comment"
}
```

Válasz formátuma:

```
{
    "Sikeres létrehozás"
    "id": 4
}
```

1. **Hibakezelés**: 400: "missing data" - Hiányos paraméterek.
2. **Hibakezelés**: 400: "Ez a barber már foglalt az adott időintervallumban" - Már foglalt a barber az új időpont idejében.
3. **Hibakezelés:** 500: “Szerverhiba” - Szerverhiba.

POST /appointmentUserPost

Leírás:Időpont lefoglalása. .

Várható bemenetek:

- **Header**:

  Authorization: Bearer {token}

- **Body**

```
{
    "id": 4,
    "barberID": 4,
    "serviceID": 4,
    "start_time": "2026-03-08 12:00:00",
    "end_time": "2026-03-08 12:30:00",
    "comment": "asd"
}
```

Válasz formátuma:

```
{
    "Foglalás sikeres"
    "id": 4
}
```

1. **Hibakezelés**: 400: "missing data" - Hiányos paraméterek.
2. **Hibakezelés**: 400: "Ez az időpont már foglalt” - Már foglalt időpont.
3. **Hibakezelés**: 400: "Invalid date format" - Rossz dátum formátum.
4. **Hibakezelés:** 500: “Szerverhiba” - Szerverhiba.

PUT /appointmentUpdate/:id

Leírás: Időpont adatainak módosítása.

Várható bemenetek:

- **Header**:
  Authorization: Bearer {token}
- **Body**

```
{
    "barberID": 5,
    "serviceID": 5,
    "start_time": "2026-03-24 17:00:00",
    "end_time": "2026-03-24 17:30:00",
    "comment": "sssss"
}
```

Válasz formátuma:

```
{
    "Sikeres Módosítás"
    "id": 3
}
```

1. **Hibakezelés**: 400: "Nincs módosítandó adat" - Hiányos paraméterek.
2. **Hibakezelés:** 400 “Invalid ID” - Rossz vagy nem létező ID
3. **Hibakezelés:** 400 “ Ez a barber már foglalt” - Valamilyen ütközés van a foglalás módosításában
4. **Hibakezelés:** 401 “No token” - Nem létező vagy lejárt token
5. **Hibakezelés:** 404 “Időpont nem található - Nem létező időpont
6. **Hibakezelés**: 500 “Szerverhiba” - Szerverhiba

Delete /appointmentDelete

Leírás: Időpont törlése.

Várható bemenetek:

- **Header**:
  Authorization: Bearer {token}
- **Body**

Válasz formátuma:

```
{
    "Sikeres Törlés"
    "id": 3
}
```

1. **Hibakezelés:** 400 “Invalid ID” - Rossz vagy nem létező ID
2. **Hibakezelés:** 401 “No token” - Nem létező vagy lejárt token
3. **Hibakezelés:** 404 “Nincs ilyen felhasználó - Nem létező felhasználó
4. **Hibakezelés**: 500 “Szerverhiba” - Szerverhiba

1.3.5. Munkaidő / Nyitvatartás kezelő API

Az API célja a borbélyok munkaidejének (nyitvatartásának) kezelése. Lehetővé teszi a borbélyok számára, hogy megadják, módosítsák és töröljék a saját munkaidejüket.

A rendszer biztosítja, hogy egy adott időintervallum ne legyen átfedésben más munkaidőkkel.

a. Általános információk

- **Bázis URL**: https://api.slickbarber.hu
- **Protokoll**: HTTPS
- **Formátum**: JSON
- **Autentikáció**: Bearer Token (JWT)

b. Hibakódok

Hibakód	Leírás
200/201	Sikeres kérés
400	Helytelen kérés
401	Jogosultság hiánya
404	Nem található
500	Szerverhiba

c. Végpontok

GET /workhoursGet

Leírás: Az összes munkaidő adatainak lekérdezése.

Várható bemenetek: **Header**: Authorization: Bearer {token}

Válasz formátuma:

```
{
    "id": 1,
    "barberID": 1,
    "dayOfWeek": 3,
    "start_time": "10:00:00",
    "end_time": "19:30:00"
  },
  {
    "id": 2,
    "barberID": 2,
    "dayOfWeek": 5,
    "start_time": "08:00:00",
    "end_time": "15:30:00"
  }
```

Hibakezelés: 401: "Unauthorized" - Hibás vagy hiányzó token.

GET /workhoursMy

Leírás: A borbély saját munkarendjének lekérdezése.

Várható bemenetek: **Header**: Authorization: Bearer {token}

Válasz formátuma:

```
{
  "id": 1,
  "barberID": 1,
  "dayOfWeek": 3,
  "start_time": "10:00:00",
  "end_time": "19:30:00"
},
{
  "id": 2,
  "barberID": 1,
  "dayOfWeek": 4,
  "start_time": "08:00:00",
  "end_time": "15:30:00"
},
{
  "id": 3,
  "barberID": 1,
  "dayOfWeek": 5,
  "start_time": "11:00:00",
  "end_time": "20:30:00"
}
```

Hibakezelés: 401: "Unauthorized" - Hibás vagy hiányzó token.

POST /workhoursPost

Leírás: Új munkaidő hozzáadása.

Várható bemenetek:

- **Header**:
  Authorization: Bearer {token}
- **Body**:

```
{
    "dayOfWeek": 1,
    "start_time": "12:00:00",
    "end_time": "21:30:00"
}
```

Válasz formátuma:

```
{
    "Sikeres Létrehozás"
    "id": 1
}
```

1. **Hibakezelés**: 400: "Hiányzó adatok" - Hiányzó adatok.
2. **Hibakezelés**: 400: "Már létező munkaidő" - Már létezik azon a napon/ órákban munkaideje a borbélynak.
3. **Hibakezelés**: 500: "Szerverhiba" - Szerverhiba.

PUT/workhoursUpdate

Leírás: Munkaórák adatainak módosítása.

Várható bemenetek:

- **Header**:
  Authorization: Bearer {token}
- **Body**

```
{
    "dayOfweek": 2
    "start_time": "13:00:00",
    "end_time": "22:30:00"
}
```

Válasz formátuma:

```
{
    "Sikeres Módosítás"
    "id": 3
}
```

1. **Hibakezelés**: 400: "Nincs módosítandó adat" - Hiányos paraméterek.
2. **Hibakezelés:** 400 “Invalid ID” - Rossz vagy nem létező ID
3. **Hibakezelés:** 400 “ Ez a barber már foglalt” - Valamilyen ütközés van a foglalás módosításában
4. **Hibakezelés:** 401 “Hiányzó jogosultság” - Nem létező vagy lejárt token
5. **Hibakezelés:** 404 “Nincs ilyen munkaóra” - Nem létező időpont
6. **Hibakezelés**: 500 “Szerverhiba” - Szerverhiba

Delete/workhoursDelete

Leírás: Munkaóra törlése.

Várható bemenetek:

- **Header**:
  Authorization: Bearer {token}
- **Body**

Válasz formátuma:

```
{
    "Sikeres Törlés"
    "id": 3
}
```

1. **Hibakezelés:** 400 “Invalid ID” - Rossz vagy nem létező ID
2. **Hibakezelés:** 401 “Hiányzó jogosultság” - Nem létező vagy lejárt token
3. **Hibakezelés:** 404 “Nincs ilyen munkaidő - Nem létező munkaidő
4. **Hibakezelés**: 500 “Szerverhiba” - Szerverhiba

1.3.6. Middleware logikája

a. **Leírás:** A rendszer nem bízik meg a kliens által küldött adatokban. Minden kérés előtt lefut a hitelesítő réteg (Auth.js), amely a kérés fejlécében (Authorization: Bearer) található JWT tokent elemzi és validálja.

b. **Auth() (Általános hitelesítés):** Ez az alapvető middleware dekódolja a tokent, és csak a bejelentkezés tényét ellenőrzi. Olyan végpontoknál alkalmazzuk (pl. időpont törlése), ahol mind a borbélyoknak, mind a vendégeknek jogosultsága van a művelet elvégzésére.

c. **AuthUser() / AuthBarber() (Szerepkör alapú ellenőrzés):** A token érvényessége mellett a dekódolt payload alapján ellenőrzik a felhasználó szerepkörét (role). A vendégek nem férhetnek hozzá a borbélyok naptárához, és fordítva.

d. **AuthAdmin() (Kiemelt biztonság):** Nemcsak a tokent nézi meg, hanem a kérés beérkezésekor visszakérdez az adatbázisba is (findByPk), hogy ellenőrizze, az adott fodrász valóban rendelkezik-e még isAdmin jogosultsággal. Ez megakadályozza, hogy valaki "meghackelje" a rendszergazdai funkciókat egy régi, de még érvényes tokennel, ha közben a jogait az adatbázisban visszavonták.

1.4. Reszponzív weboldal

1.4.1. Technológiai architektúra:

A kliensoldali webes felület a modern webes szabványokra épül, biztosítva a gyors betöltést és a platformfüggetlenséget.

- **HTML5 & CSS3:** A tartalom szerkezetéért és a vizuális formázásért felel.
- **Bootstrap 5:** Frontend keretrendszerként biztosítja a reszponzív rácsrendszert (Grid) és az előre gyártott UI komponenseket.
- **Flexbox & Media Queries:** Egyedi elrendezésekhez és a különböző kijelzőméretekre (mobil, tablet, asztali) való optimalizáláshoz.

1.4.2. Frontend komponensek és stílus:

a. Layout és Grid rendszer

A weboldal felépítése a Bootstrap 12 oszlopos rácsrendszerét használja. A szekciók (pl. Rólunk, Árak, Csapat) rugalmas konténerekbe (container) vannak foglalva, amelyek automatikusan igazodnak a képernyő szélességéhez.

b. Glassmorphism implementáció

A modern vizuális megjelenés érdekében a kártyák és modálok „glassmorphism” stílust kaptak.

- Technikai megvalósítás: background-color: rgba(..., 0.5) (félig átlátszó háttér) és a backdrop-filter: blur(...) CSS tulajdonság alkalmazása.
- Vizuális elemek: Lekerekített sarkok (border-radius), finom keret és mélységérzetet adó árnyékok.

c. **Kliensoldali állapotkezelés**

A rendszer a felhasználó munkamenetét a böngésző sessionStorage objektumában kezeli.

- **Autentikációs token**: Bejelentkezés után a szervertől kapott JWT token ide kerül tárolásra.
- **Hozzáférés-ellenőrzés**: A védett oldalak (pl. Foglalás, Profil) betöltésekor egy script ellenőrzi a token meglétét. Hiány esetén a rendszer automatikusan visszairányít a főoldalra.

1.4.3. Dinamikus modulok logikája:

a. **Interaktív árlista**

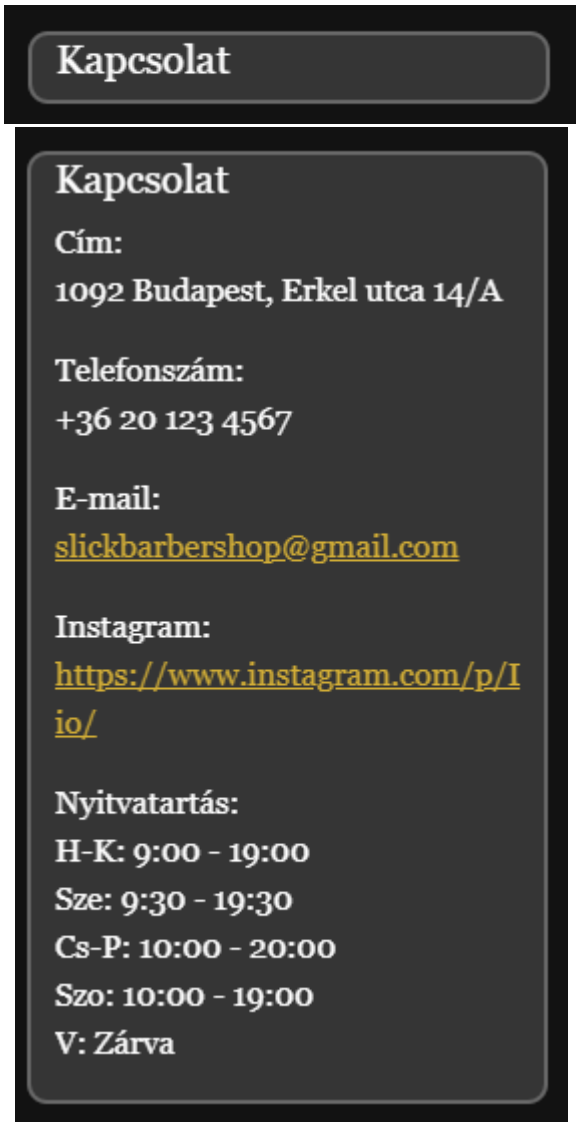
Az árak megjelenítése tab-alapú (füles) rendszerben történik.

- **Logika**: A prices.js fájl kezeli a kattintási eseményeket. Minden gomb rendelkezik egy data-tab attribútummal, amely megegyezik a hozzá tartozó tartalom id-jával.
- **Osztályváltás**: Kattintáskor a script eltávolítja az active osztályt az összes tabról és tartalomról, majd hozzáadja a kiválasztotthoz.

b. Kapcsolat modul

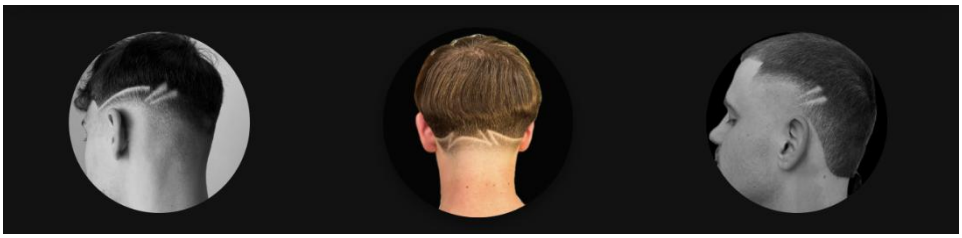
Az elérhetőségi adatok egy contact-box elemben jelennek meg.

- **Működés**: A JavaScript fájl az „open” osztály ki- és bekapcsolásával vezérli a modul lenyitását és visszazárását, biztosítva, hogy görgetés közben is elérhető maradjon.



c. Munkáink szekció

- **Interakció**: A képek alapértelmezetten fekete-fehér szűrővel rendelkeznek.
- **Technológia**: CSS filter: grayscale(100%) használata, amely a :hover esemény hatására (egér rámutatás) grayscale(0%)-ra vált, visszaadva az eredeti színeket.



1.4.4. Foglalási folyamat technikai leírása:

a. Többlépcsős űrlapkezelés

A foglalás folyamata logikailag elkülönített blokkokra (booking-step) van bontva.

1. Fodrász kiválasztása.
2. Szolgáltatás dinamikus betöltése API-n keresztül.
3. Dátum és szabad időpontok lekérése.
4. Véglegesítés és adatok elküldése.

b. Naptár komponens implementáció

Egy egyedi JavaScript alapú naptár generálja az adott hónap napjait. A script blokkolja a múltbeli dátumokat és a vasárnapokat.

c. Validációk és hibaüzenetek

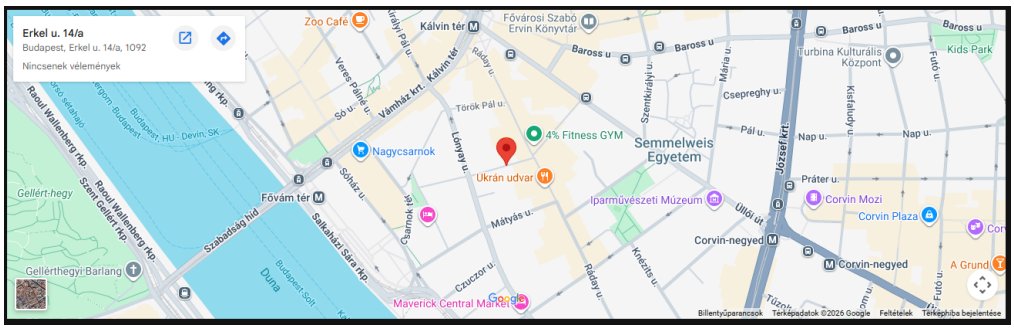
A beküldés előtt a rendszer ellenőrzi, hogy minden kötelező mező (required) ki van-e töltve. Hiba vagy sikeres foglalás esetén a felhasználó böngésző-szintű alert értesítést kap.

1.4.5. Külső szolgáltatások integrációja:

a. Google Maps API / Iframe

Az üzlet pontos helyszínét egy beágyazott Google Maps iframe mutatja.

- **Optimalizálás**: Alkalmazva van a loading="lazy" attribútum, így a térkép csak akkor töltődik be, ha a felhasználó a szekcióhoz görget, javítva ezzel az oldal sebességét.



b. Backend kommunikáció

A frontend és a backend közötti adatforgalom REST API-n keresztül zajlik.

- **Adatküldés**: Például sikeres foglaláskor a rendszer egy POST /appointmentUserPost hívást indít a szerver felé a gyűjtött adatokkal.

1.5. Fodrászszalon Kliens Alkalmazás

1.5.1. Specifikáció

A kliens alkalmazás célja, hogy a fodrászok (Barber) és az üzletvezetők (Admin) egy modern, gyors és platformfüggetlen felületen tudják kezelni a mindennapi feladatokat. Az alkalmazás nem végez közvetlen adatbázis-műveleteket, minden adatot a Node.js REST API-n keresztül ér el.

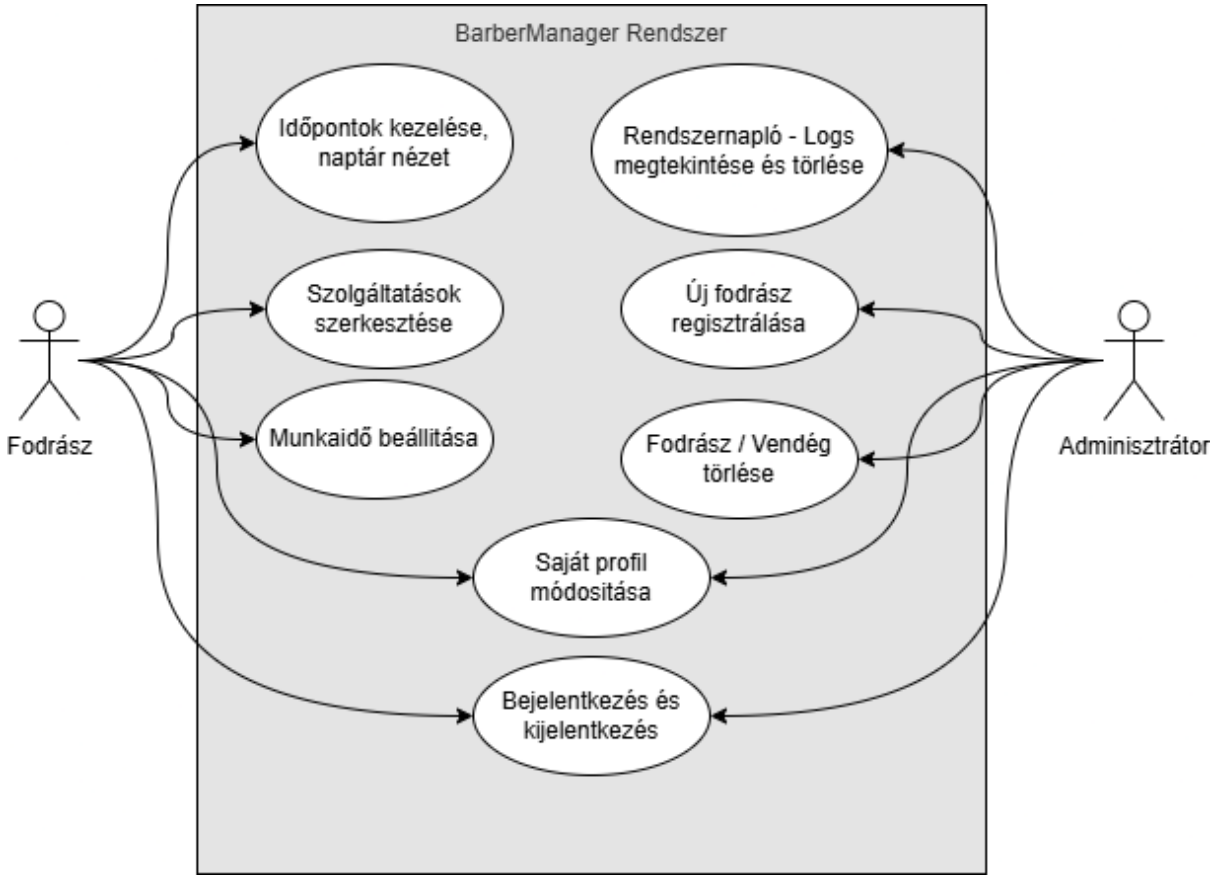
1.5.2. Funkcionális követelmények

- **Autentikáció**: Bejelentkezés email, név, és jelszó alapján. A sikeres belépés után kapott JWT tokent a rendszer eltárolja.
- **Időpontok kezelése**: A fodrász láthatja a hozzá tartozó foglalásokat. Asztali nézetben egy interaktív, heti bontású naptár, míg mobil nézetben egy napi lista jelenik meg.
- **Szolgáltatások és Munkaidő**: A fodrászok felvihetik és árazhatják saját szolgáltatásaikat, valamint napokra lebontva beállíthatják a munkaidejüket.
- **Adminisztráció**: Az "Admin" jogosultsággal rendelkező fodrászok számára elérhető egy külön panel, ahol új fodrászokat regisztrálhatnak, valamint törölhetnek meglévő fodrászokat és vendégeket.
- **Naplózás**: Az Adminok megtekinthetik a rendszerben történt eseményeket (foglalások, törlések), és egy gombnyomással törölhetik a 30 napnál régebbi bejegyzéseket.
- **Profil testreszabása**: A fodrászok saját profilképet tölthetnek fel (max. 5MB) és bemutatkozó szöveget írhatnak a weboldalra.

1.5.3. Nem-funkcionális követelmények

- **Reszponzivitás és Hordozhatóság**: A kód C# Avalonia UI keretrendszerben íródott, így ugyanaz az üzleti logika fut a Windows asztali és az Android mobil verziókon.
- **Aszinkron működés**: A hálózati (API) hívások nem fagyasztják meg a grafikus felületet, az adatok letöltése aszinkron (async/Task) módon történik.

1.5.4. Használati Eset Diagram



1. ábra: A C# kliens alkalmazás használati eset diagramja

1.5.5. Felhasználói történetek

Bejelentkezés

I WANT TO: Bejelentkezni akarok a rendszerbe fodrászként vagy adminként.

GIVEN	Elindítottam az alkalmazást, és a bejelentkezési képernyőn vagyok.
WHEN	Kitöltöm a név, email és jelszó mezőket helyes adatokkal, majd a “Bejelentkezés” gombra kattintok.
THEN	Az ApiService validálja az adatokat a szerverrel, elmenti a JWT tokent, az alkalmazás letölti a profilomat, és a jogosultságomnak (Admin vagy Barber) megfelelő főképernyőre navigál.

Kijelentkezés

I WANT TO: Kiszeretnék jelentkezni a fiókból.

GIVEN	Be vagyok jelentkezve és látom az oldalsó menüsort.
WHEN	A menü alján található piros "Kijelentkezés" gombra kattintok.
THEN	Az alkalmazás törli a JWT tokent, kiüríti a bejelentkezett fodrász adatait és visszairányít a LoginView képernyőre.

Heti naptár megtekintése (Asztali nézeten)

I WANT TO: Megszeretném nézni a heti naptárat asztali gépen.

GIVEN	Asztali gépen fodrászként be vagyok jelentkezve
WHEN	Az "Időpontok" menüpontra kattintok.
THEN	Az alkalmazás letölti a heti foglalásaimat és a DrawUI metódus segítségével színes kártyákként elhelyezi őket a heti naptárrácsban.

Napi foglalások kezelése (Mobil nézet)

I WANT TO: Megszeretném nézni a heti naptárat telefonon.

GIVEN	Mobil eszközön fodrászként be vagyok jelentkezve
WHEN	A naptár gombra rányomok alul.
THEN	A ViewLocator felismeri a platformot, és a napi listás nézetet (AppointmentsViewMobile) tölti be, ahol csak a mai foglalásaim látszódnak.

Időpont részleteinek megtekintése

I WANT TO: Megszeretném nézni egy foglalásnak a részleteit.

GIVEN	Látok egy foglalást a naptárban.
WHEN	A kártyán lévő három pontra (Részletek) kattintok.
THEN	Megnyílik egy modális ablak, amely megjeleníti a vendég nevét, telefonszámát, email címét és a foglaláshoz fűzött megjegyzést.

Időpont befejezettnek jelölése

I WANT TO: Megszeretném jelölni teljesítettnek a foglalást..

GIVEN	Van egy aktív (booked) foglalásom a naptárban.
WHEN	A kártyán lévő "pipa" gombra kattintok.
THEN	Az alkalmazás completed státuszra frissíti az időpontot a szerveren, a kártya színe zöldre változik, a pipa gomb pedig eltűnik.

Időpont lemondása

I WANT TO: Leszeretnék mondani egy foglalást

GIVEN	Megnyitottam egy foglalás részleteit.
WHEN	Az "Időpont Lemondása" gombra kattintok.
THEN	A rendszer canceled státuszra állítja a foglalást, a naptárban a kártya szürke lesz, a vendég neve pedig áthúzva jelenik meg.

Saját szolgáltatások listázása

I WANT TO: Megszeretném nézni a szolgáltatásaimat.

GIVEN	A "Szolgáltatások" menüpontra kattintok.
WHEN	Betöltődik a ServicesView.
THEN	Az alkalmazás lekéri a profilomhoz rendelt összes szolgáltatást (vágás, festés stb.) és kártyás elrendezésben megjeleníti az árukkal együtt.

Új szolgáltatás hozzáadása

I WANT TO: Szeretnék egy új szolgáltatást létrehozni

GIVEN	A szolgáltatások listájánál vagyok.
WHEN	Az "Új hozzáadása" “+” gombra kattintok, kitöltöm a nevet, leírást, árat és időtartamot, majd mentem.
THEN	A rendszer POST kéréssel létrehozza a szolgáltatást, majd frissíti a listát.

Szolgáltatás módosítása

I WANT TO: Szeretném módosítani az egyik szolgáltatásom

GIVEN	Egy meglévő szolgáltatás kártyáján a szerkesztés (ceruza) ikonra kattintok.
WHEN	Átírom a szolgáltatás árát és a "Mentés" gombra kattintok.
THEN	Az alkalmazás PUT kéréssel frissíti az adatokat a szerveren és visszatér a lista nézethez.

Szolgáltatás törlése

I WANT TO: Szeretném törölni az egyik szolgáltatásom

GIVEN	Egy meglévő szolgáltatás kártyáján a törlés (kuka) ikonra kattintok.
WHEN	Megerősítem a törlési szándékot.
THEN	Az alkalmazás törli a szolgáltatást a rendszerből, és az eltűnik a listából.

Heti munkaidő beállítása

I WANT TO: Szeretném beállítani a heti munkaidőmet

GIVEN	A "Munkaidő" menüpontban vagyok.
WHEN	Beállítom a naponkénti kezdő és záró időpontokat, majd a "Beállítások mentése" gombra kattintok.
THEN	A rendszer ciklusban végigküldi a napok adatait az API-nak, és sikeres mentés után visszajelzést ad a felhasználónak.

Saját profil adatainak módosítása

I WANT TO: Szeretném módosítani a saját profilom.

GIVEN	A "Profilom" menüpontban vagyok.
WHEN	Megváltoztatom a nevemet vagy a telefonszámomat, majd a mentésre kattintok.
THEN	A profilom frissül az adatbázisban, és a menüsorban lévő nevem is azonnal átíródik az újra.

Jelszóváltás

I WANT TO: Szeretném módosítani a jelszavam.

GIVEN	A profil szerkesztése oldalon vagyok.
WHEN	Megadom az új jelszót két alkalommal, majd mentem.
THEN	Ha a két jelszó egyezik, a kliens elküldi a kérést, a backend pedig titkosítva (bcrypt) elmenti az új jelszót.

Adminisztrátori (Admin) funkciók

Adminisztrációs felület elérése

I WANT TO: Megszeretném nézni az adminisztrációs felületet

GIVEN	Admin jogosultsággal jelentkeztem be.
WHEN	Megtekintem az oldalsó menüt.
THEN	Megjelenik a "Felhasználók kezelése" és a "Napló" menüpont, ami sima fodrászoknak nem látható.

Új fodrász regisztrálása

I WANT TO: Szeretnék regisztrálni egy új fodrászt.

GIVEN	A "Kezelés" (AdminPanel) oldalon vagyok.
WHEN	Kitöltöm az új fodrász adatait (név, email, jelszó, tel), bepipálom az Admin jogot ha szükséges, és a regisztrálásra kattintok.
THEN	Az alkalmazás létrehozza az új fiókot, és az azonnal megjelenik a regisztrált fodrászok listájában.

Fodrász fiók törlése

I WANT TO: Szeretnék törölni egy meglévő fodrászt.

GIVEN	A regisztrált fodrászok listáját böngészem.
WHEN	Egy fodrász neve melletti "Törlés" gombra kattintok.
THEN	A rendszer véglegesen törli a fodrászt az adatbázisból, és frissíti a listát.

Regisztrált vendég (User) törlése

I WANT TO: Szeretnék törölni egy regisztrált vendéget.

GIVEN	Az Admin panelen a "Regisztrált Vendégek" listájához görgetek.
WHEN	Egy vendég melletti "Törlés" gombra kattintok.
THEN	Az alkalmazás törli a felhasználót, aminek hatására a backend automatikusan lemondja az adott vendég összes jövőbeli időpontját is.

Rendszernapló (Logs) megtekintése

I WANT TO: Szeretném megtekinteni a rendszernaplót.

GIVEN	A "Napló" menüpontra kattintok.
WHEN	Betöltődik a LogsView.
THEN	Látom az összes rendszerszintű eseményt (ki, mikor, mit módosított vagy törölt) időrendi sorrendben.

Napló takarítása (Cleanup)

I WANT TO: Szeretném törölni a 30 napnál régebbi naplózásokat.

GIVEN	A napló nézetben vagyok.
WHEN	A "30 napnál régebbi törlése" gombra kattintok.
THEN	Az alkalmazás utasítja a szervert a régi logok törlésére, majd frissíti a nézetet.

Időpont rögzítése

I WANT TO: Új időpontot rögzíteni egy vendégnek a naptárban

GIVEN	Fodrászként a "Naptár" nézeten vagyok.
WHEN	Rákattintok az "Új foglalás" gombra, a felugró ablakban kiválasztom a vendéget, a szolgáltatást, a dátumot, majd a legördülő menüből egy szabad időpontot, és a "Mentés" gombra nyomok.
THEN	Az alkalmazás elküldi a POST kérést az API-nak. Sikeres válasz esetén az űrlap bezáródik, és az új időpont színes kártyaként megjelenik a naptárban.

1.5.6. Szoftver Architektúra és Tervezés

a. MVVM Architektúra

Az alkalmazás az MVVM (Model-View-ViewModel) tervezési mintát követi a CommunityToolkit.Mvvm csomag felhasználásával. Ez garantálja a grafikus felület és az üzleti logika éles szétválasztását.

- **Model**: A Models mappában találhatók (pl. Barber.cs, Appointment.cs). Ezek egyszerű C# osztályok, amelyek a JSON adatok struktúráját írják le. A [JsonPorpertyName] attribútumok felelnek a pontos adatkötésért az API válaszokkal.
- **View**: A Views mappában léve .axaml fájlok (pl. AppointmentsView.axaml). Csak UI elemeket (gombok, szövegdobozok, naptár rácsok) tartalmaznak. Minden asztali nézetnek megvan a mobilos megefelője (pl. AppointmentsViewMobile.axaml).
- **ViewModel**: Ezek az osztályok hívják meg a backendet, és tárolják a képernyő állapotát. Az [ObservableProperty] attribútum gondoskodik róla hogy ha egy változó (pl. hibaüzenet vagy lista) értéke megváltozik, a grafikus felület azonnal frissüljön. A gombok kattintásait a [RelayCommand] attribútummal ellátott metódusok kezelik.

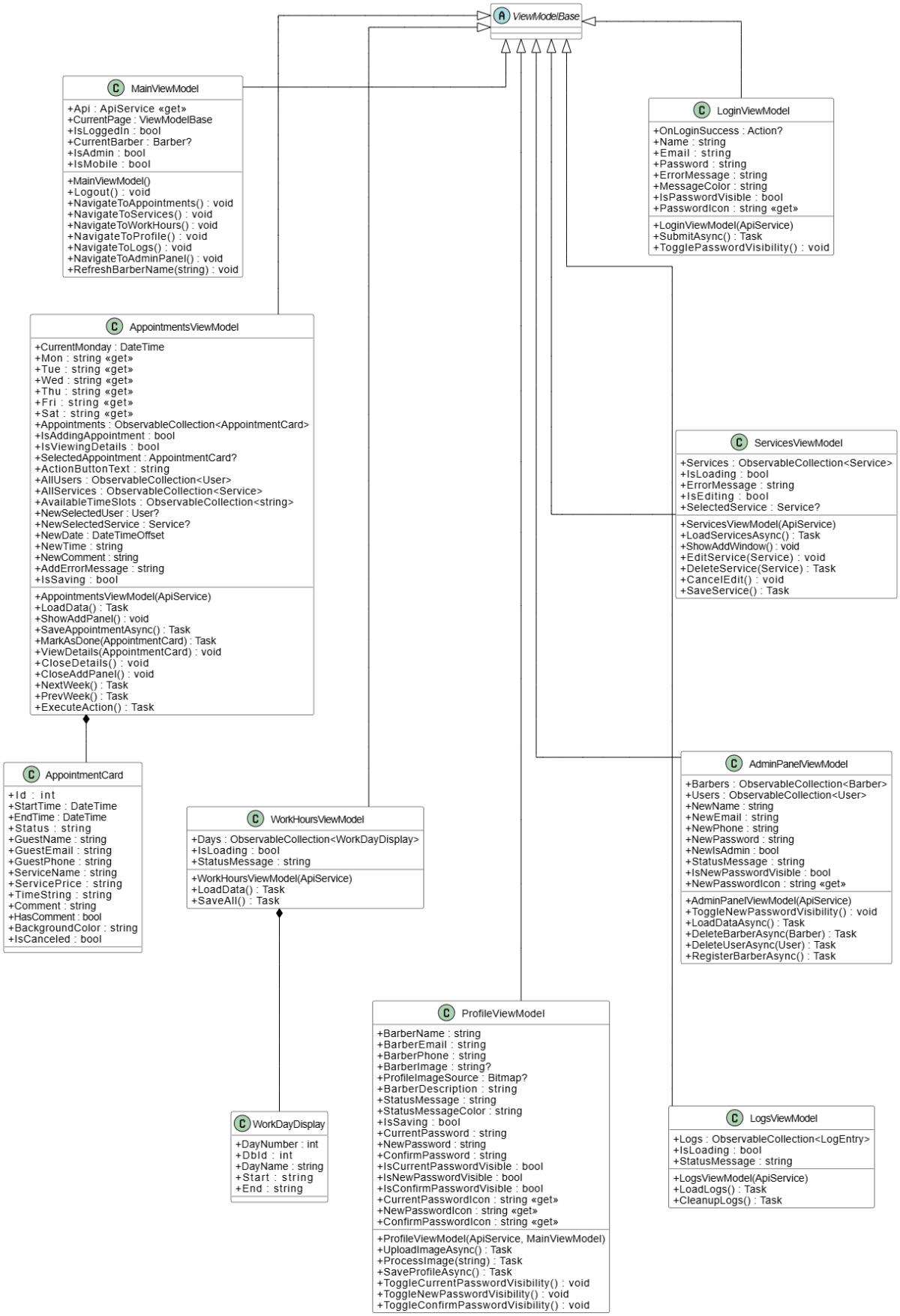
b. REST API Kommunikáció

A hálózati kommunikációt egy központosított szolgáltatás, az ApiService.cs végzi HttpClient segítségével.

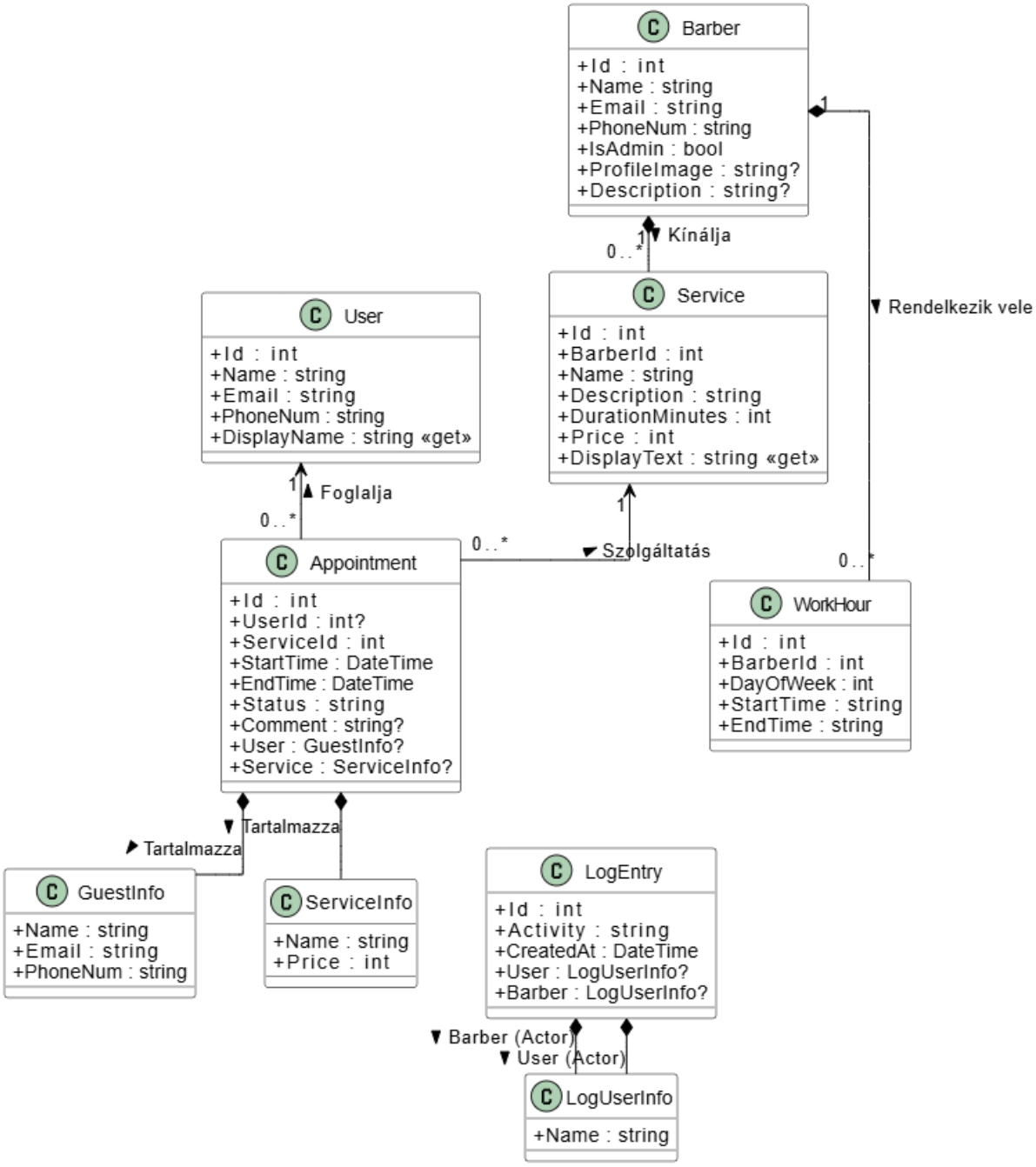
Az osztály legfőbb funkciói:

- **Token Kezelés**: A bejelentkezés során kapott JWT tokent egy privát változóban tárolja, és a SetAuthorizationHeader() metódussal minden további kérés fejlécébe kerül.
- **Egyedi JSON Konverterek**: Mivel a backend időnként más típusban (pl. TinyInt/Boolean, Date formátumok) küldi az adatokat, egyedi konverteket implementáltam (pl. LocalTimeConverter, IntToBoolConverter) a hibátlan deserializáció érdekében.

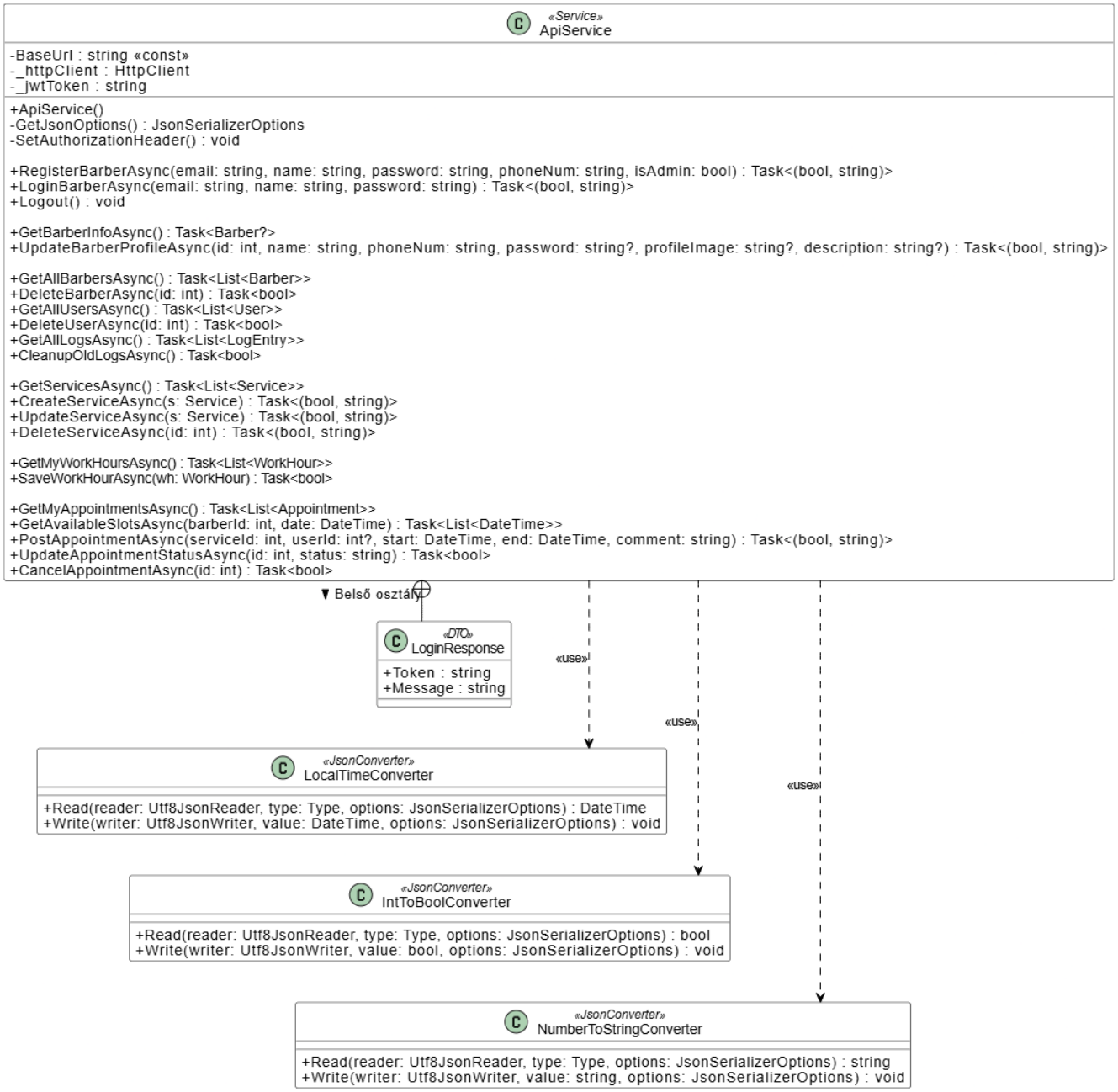
c. ViewModels Osztálydiagram


ViewModelBase
MainViewModel
+Api : ApiService «get»
+CurrentPage : ViewModelBase
+IsLoggedIn : bool
+CurrentBarber : Barber?
+IsAdmin : bool
+IsMobile : bool
+MainViewModel()
+Logout() : void
+NavigateToAppointments() : void
+NavigateToServices() : void
+NavigateToWorkHours() : void
+NavigateToProfile() : void
+NavigateToLogs() : void
+NavigateToAdminPanel() : void
+RefreshBarberName(string) : void
LoginViewModel
+OnLoginSuccess : Action?
+Name : string
+Email : string
+Password : string
+ErrorMessage : string
+MessageColor : string
+IsPasswordVisible : bool
+PasswordIcon : string «get»
+LoginViewModel(ApiService)
+SubmitAsync() : Task
+TogglePasswordVisibility() : void
AppointmentsViewModel
+CurrentMonday : DateTime
+Mon : string «get»
+Tue : string «get»
+Wed : string «get»
+Thu : string «get»
+Fri : string «get»
+Sat : string «get»
+Appointments : ObservableCollection<AppointmentCard>
+IsAddingAppointment : bool
+IsViewingDetails : bool
+SelectedAppointment : AppointmentCard?
+ActionButtonText : string
+AllUsers : ObservableCollection<User>
+AllServices : ObservableCollection<Service>
+AvailableTimeSlots : ObservableCollection<string>
+NewSelectedUser : User?
+NewSelectedService : Service?
+NewDate : DateTimeOffset
+NewTime : string
+NewComment : string
+AddErrorMessage : string
+IsSaving : bool
+AppointmentsViewModel(ApiService)
+LoadData() : Task
+ShowAddPanel() : void
+SaveAppointmentAsync() : Task
+MarkAsDone(AppointmentCard) : Task
+ViewDetails(AppointmentCard) : void
+CloseDetails() : void
+CloseAddPanel() : void
+NextWeek() : Task
+PrevWeek() : Task
+ExecuteAction() : Task
ServicesViewModel
+Services : ObservableCollection<Service>
+IsLoading : bool
+ErrorMessage : string
+IsEditing : bool
+SelectedService : Service?
+ServicesViewModel(ApiService)
+LoadServicesAsync() : Task
+ShowAddWindow() : void
+EditService(Service) : void
+DeleteService(Service) : Task
+CancelEdit() : void
+SaveService() : Task
AppointmentCard
+Id : int
+StartTime : DateTime
+EndTime : DateTime
+Status : string
+GuestName : string
+GuestEmail : string
+GuestPhone : string
+ServiceName : string
+ServicePrice : string
+TimeString : string
+Comment : string
+HasComment : bool
+BackgroundColor : string
+IsCanceled : bool
WorkHoursViewModel
+Days : ObservableCollection<WorkDayDisplay>
+IsLoading : bool
+StatusMessage : string
+WorkHoursViewModel(ApiService)
+LoadData() : Task
+SaveAll() : Task
AdminPanelViewModel
+Barbers : ObservableCollection<Barber>
+Users : ObservableCollection<User>
+NewName : string
+NewEmail : string
+NewPhone : string
+NewPassword : string
+NewIsAdmin : bool
+StatusMessage : string
+IsNewPasswordVisible : bool
+NewPasswordIcon : string «get»
+AdminPanelViewModel(ApiService)
+ToggleNewPasswordVisibility() : void
+LoadDataAsync() : Task
+DeleteBarberAsync(Barber) : Task
+DeleteUserAsync(User) : Task
+RegisterBarberAsync() : Task
ProfileViewModel
+BarberName : string
+BarberEmail : string
+BarberPhone : string
+BarberImage : string?
+ProfileImageSource : Bitmap?
+BarberDescription : string
+StatusMessage : string
+StatusMessageColor : string
+IsSaving : bool
+CurrentPassword : string
+NewPassword : string
+ConfirmPassword : string
+IsCurrentPasswordVisible : bool
+IsNewPasswordVisible : bool
+IsConfirmPasswordVisible : bool
+CurrentPasswordIcon : string «get»
+NewPasswordIcon : string «get»
+ConfirmPasswordIcon : string «get»
+ProfileViewModel(ApiService, MainViewModel)
+UploadImageAsync() : Task
+ProcessImage(string) : Task
+SaveProfileAsync() : Task
+ToggleCurrentPasswordVisibility() : void
+ToggleNewPasswordVisibility() : void
+ToggleConfirmPasswordVisibility() : void
WorkDayDisplay
+DayNumber : int
+DbId : int
+DayName : string
+Start : string
+End : string
LogsViewModel
+Logs : ObservableCollection<LogEntry>
+IsLoading : bool
+StatusMessage : string
+LogsViewModel(ApiService)
+LoadLogs() : Task
+CleanupLogs() : Task

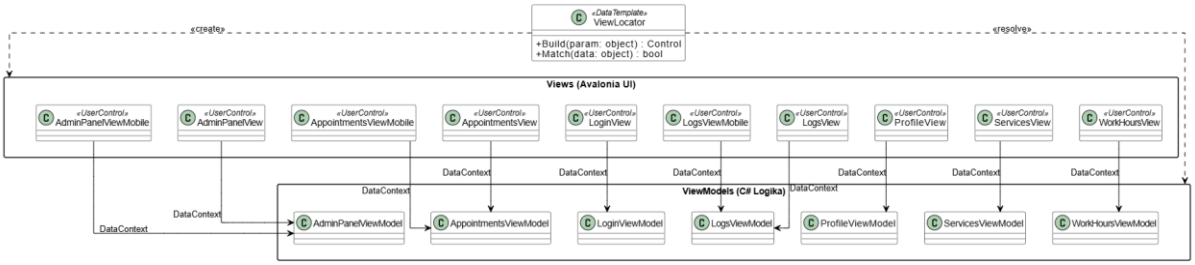
d. Models Osztálydiagram



e. Services Osztálydiagram



f. View és ViewModel Osztálydiagram

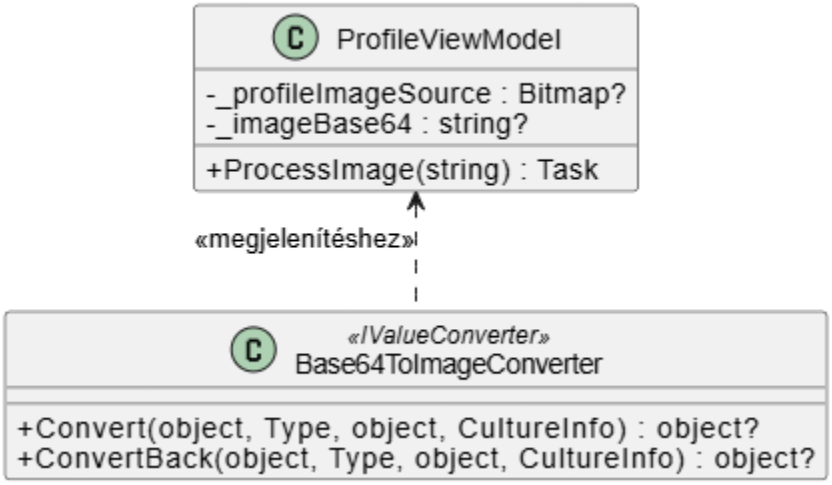


g. Multimédia kezelés és Adatkonverzió

"Az alkalmazásban a képek kezelése egy egyedi konverziós láncon megy keresztül a hatékony hálózati kommunikáció és a rugalmas megjelenítés érdekében:

- **Kliens oldali feldolgozás:** A kép kiválasztásakor a ProfileViewModel bájtszinten olvassa be a fájlt, majd Base64 formátumra kódolja. Ezzel elkerüljük a komplex multipart/form-data kéréseket, és maradunk a tiszta JSON kommunikációnál.
- **IValueConverter technológia:** A nézet (View) oldalon a Base64ToImageConverter osztályt használjuk. Ez egy speciális Avalonia/WPF interfész, amely az adatkötés (Binding) pillanatában, futásidőben alakítja vissza a szöveget Bitmap objektummá.
- **Memóriaoptimalizálás:** A konverzió során MemoryStream-et használunk, ami biztosítja, hogy ne maradjanak felesleges ideiglenes fájlok a lemezen, és a kép közvetlenül a RAM-ból töltődjön be a UI-ra."

Képek kezelése és konvertálása osztálydiagram



1.6. Tesztelési dokumentáció

1.6.1. Bevezetés

1.6.2. REST API Tesztelés

1.6.2.1. Tesztelési cél

Ez a dokumentáció az automata tesztelési folyamatokat foglalja össze. Célja a REST API végpontok (GET, POST, PUT, DELETE) helyes működésének, a jogosultságkezelésnek és a hibakezelési mechanizmusoknak az ellenőrzése.

1.6.2.2 Tesztelt modulok

- Borbély Regisztráció
- Borbély Bejelentkezés
- Borbély saját adatainak lekérdezése
- Borbély saját adatainak módosítása
- Borbély törlése
- Borbély saját szolgáltatásának létrehozása
- Borbély saját szolgáltatásainak lekérdezése
- Borbély saját szolgáltatásainak módosítása
- Borbély saját szolgáltatásainak törlése
- Borbély saját időpontjainak létrehozása
- Borbély saját időpontjainak lekérdezése
- Borbély saját időpontjainak módosítása
- Borbély saját időpontjainak törlése
- Borbély saját beosztásának létrehozása
- Borbély saját beosztásának lekérdezése
- Borbély saját beosztásának módosítása
- Borbély saját beosztásának törlése
- Borbély logjainak lekérdezése
- Borbély vendégeinek adatainak lekérdezése
- Minden vendég adatainak lekérdezése
- Minden Szolgáltatás adatainak lekérdezése
- Minden beosztás adatainak lekérdezése
- Minden Borbély adatainak lekérdezése
- Minden lognak lekérdezése
- Vendég Regisztráció
- Vendég Bejelentkezés

- Vendégek adatainak lekérdezése
- Időpont foglalás
- Időpont lemondás
- Vendég borbélya adatainak megtekintése
- Vendég adatainak módosítása
- Vendég törlése

1.6.2.3. Tesztkörnyezet

- **Operációs rendszer**: Windows 10
- **Fejlesztői környezet**: Visual Studio Code
- **Adatbázis kiszolgáló:** MySQL Server
- **Adatbázis kezelő**: MySQL - phpMyAdmin
- **Futtató környezet:** Node.js
- **Tesztelési eszközök**: Automata tesztelés, Jest, Supertest

1.6.2.4 Automata tesztesetek

Az automata tesztelés szkriptek strukturált formában lettek megírva a következő módon:

ID	Modul / Funkció	Leírás / Tesztelt esetek	Input adatok (Példa)	Elvárt kódok	Eredmény
001	**barberReg**	Sikeres reg., hiányzó adatok és szerverhiba.	email, name, password, phoneNum, isAdmin	201, 400, 500	Pass
002	**barberLogin**	Belépés, hiányzó adatok és kényszerített hiba.	email, password, isAdmin	201, 400, 500	Pass
003	**userReg**	Vendég reg., rossz formátum és üres body.	email, name, password, phoneNum	201, 400, 500	Pass

004	**userLogin**	Vendég login, érvénytelen adatok és hiba teszt.	email, name, password	200, 400, 500	Pass
005	**servicesPost**	Új szolgáltatás, auth és adatvalidáció.	name, description, duration_minutes, price	200, 400, 401	Pass
006	**workhoursPost**	Munkaidő rögzítés, hiányzó mezők és formátum.	dayOfWeek, start_time, end_time	200, 400, 401	Pass
007	**appointmentPost**	Időpont (borbély), kötelező adatok ellenőrzése.	userID, barberID, serviceID, start_time...	200, 400, 401	Pass
008	**appointmentUser Post**	Vendég foglalása, token és hibás ID kezelése.	barberID, serviceID, start_time, status	200, 400, 401	Pass

009	**barberGet**	Borbély saját profil lekérése auth-al.	Authorization: Bearer {barberToken}	200, 401	Pass
010	**userGet**	Vendég saját adat lekérése tokennel.	Authorization: Bearer {userToken}	200, 401	Pass
011	**usersAll**	Összes user listázása admin joggal.	Authorization: Bearer {barberToken}	200, 401	Pass
012	**servicesGet**	Elérhető szolgáltatások listázása.	Nincs body (GET)	200	Pass
013	**servicesMy**	Borbély saját szolgáltatásainak listázása.	Authorization: Bearer {barberToken}	200, 401	Pass

014	**workhoursGet**	Összes rögzített munkaidő lekérdezése.	Nincs body (GET)	200	Pass
015	**workhoursMy**	Borbély saját beosztásának lekérése.	Authorization: Bearer {barberToken}	200, 401	Pass
016	**appointmentMy Barber**	Borbélyhoz tartozó foglalások lekérése.	Authorization: Bearer {barberToken}	200, 401	Pass
017	**appointmentMy User**	Vendég saját foglalásainak listázása.	Authorization: Bearer {userToken}	200, 401	Pass
018	**availableSlots**	Szabad sávok keresése barber/dátum alapján.	URL: /:barberId/:date	200, 400, 500	Pass

019	**barbersAll**	Összes borbély (Admin vs. Tiltott elérés).	Authorization: Bearer {token}	200, 403	Pass
020	**logsAll**	Rendszernaplók elérése adminisztrátorként.	Authorization: Bearer {token}	200, 401, 403	Pass
021	**logsMy**	Borbély saját tevékenységi naplója.	Authorization: Bearer {barberToken}	200, 401	Pass
022	**barberUpdate**	Borbély módosítás, validáció és létezés.	name, phoneNum + URL :id	200, 400, 401, 404	Pass
023	**userUpdate**	Vendég frissítés, jogosultságok tesztje.	phoneNum + URL :id	200, 400, 403, 404	Pass

024	**servicesUpdate**	Szolgáltatás paraméterek (ár, név) módosítása.	name, price + URL :id	200, 400, 404	Pass
025	**appointmentUpdate**	Foglalás időpont vagy megjegyzés átírása.	start_time, end_time, comment + URL :id	200, 400, 404	Pass
026	**workhoursUpdate**	Munkaidő beosztás módosítása ID alapján.	dayOfWeek, end_time + URL :id	200, 400, 404	Pass
027	**appointmentDelete**	Foglalás végleges törlése a rendszerből.	URL :id	200, 400, 404	Pass
028	**servicesDelete**	Szolgáltatás eltávolítása az adatbázisból.	URL :id	200, 400, 404	Pass

029	**workhoursDelete**	Munkaidő rekord törlése.	URL :id	200, 400, 404	Pass
030	**logsCleanup**	Régi naplók takarítása (30 napon túl).	Authorization: Bearer {barberToken}	200, 401	Pass
031	**userDelete**	Felhasználó törlés és időpont lemondás.	URL :id	200, 400, 404	Pass
032	**barberDelete**	Borbély törlés, jogosultság és létezés teszt.	URL :id	200, 400, 404	Pass

1.6.2.5. Hibák nyomon követése

Az automata tesztek során észlelt hibák rögzítéséhez a következő formátumot alkalmazzuk:

Bug ID	Hiba leírása	Lépések a reprodukálás hoz	Teszt Szkript ID	Elvárt eredmény	Aktuális eredmény	Prioritás	Státusz
BUG-01	Biztonsági hiba: Jelszó szivárgás	GET /usersAll hívása Admin tokennel	50	A válaszban a password mezőnek undefined-nek kell lennie.	A JSON válasz tartalmazza a hash-elt jelszót.	Kritikus	Javítva
BUG-02	Hiányzó validáció regisztrációnál	POST /barberReg hívása telefonszám nélkül.	006	A rendszernek 400 Bad Request kódot kell küldenie.	A rendszer 500 Internal Server Error-ral elszállt.	Magas	Javítva
BUG-03	Jogosultsági rés	GET /barbersAll hívása sima User tokennel.	068	A válasznak 403 Forbidden-nek kell lennie.	A rendszer engedte a lista lekérését (200 OK).	Magas	Javítva
BUG-04	Hibás dátumkezelés	GET /availableSlots hívása "abc" dátummal.	066	400 Bad Request hibaüzenet.	Az adatbázis lekérdezés hiba miatt leállt.	Közepes	Javítva
BUG-05	Token nélküli hozzáférés	DELETE /logsCleanup hívása Authorization fejléc nélkül.	096	401 Unauthorized státuszkód.	A takarítási folyamat elindult token nélkül is.	Kritikus	Javítva

1.6.2.6. Teszt összegzés

Az automata tesztelési fázis lezárult, melynek során a rendszer összes kritikus funkciója (regisztráció, hitelesítés, adatkezelés, jogosultságkezelés) ellenőrzésre került. A tesztelési folyamat végére a forráskód stabilitása és megbízhatósága a tervezett szintre emelkedett.

1.6.3. Reszponzív weboldal tesztelés

1.6.3.1. Tesztelési cél

A manuális tesztelés során a weboldal fő funkcióinak működését ellenőrizzük különböző eszközökön és képernyőméreteken.

1.6.2.2. Tesztelt modulok

- Navigációs menüpontok működése
- Kapcsolatok szekció megnyitása kattintásra
- Fodrászok munkáinak kattintásra való színezése
- Árak tabok végigkattintása
- Foglalás gomb működése
- Bejelentkező / Regisztrációs modal működése
- Google térkép működése
- Átirányító linkek működése
- Profil oldalon a vendég adatainak módosítása, időpont lemondása, fiók törlése
- Foglalás oldalon barberek, szolgáltatások, dátum, időpont kiválasztása
- Naptár működése
- Foglalás véglegesítése
- Vissza gombok működése
- Weboldal reszponzivitása kisebb képernyőnézetnél, illetve mobil nézeten
- Adatbázisból adatok megjelenítése
- Email sikeres elküldése

1.6.2.3. Tesztkörnyezet

- **Operációs rendszer:** Windows 10

- **Fejlesztői környezet:** Visual Studio Code
- **Használt böngésző:** Google Chrome
- **Adatbázis kiszolgáló:** MySQL Server
- **Adatbázis kezelő:** MySQL - phpMyAdmin

1.6.2.4. Teszt összegzés

A manuális tesztelés során a weboldal fő funkciói minden vizsgált eszközön megfelelően működtek.

A felület reszponzívan alkalmazkodik a különböző képernyőméretekhez, és a funkciók (navigáció, foglalás, profil kezelés) minden esetben használhatók maradtak.

1.6.4. Szoftvertesztelési dokumentáció

A C# kliensalkamazás minőségbiztosítása és hibamentes működése érdekében a tesztelés két szinten valósult meg.

1.6.4.1. Manuális tesztelési dokumentáció

a. ***1.6.4.1.1. Cél meghatározása***
   A tesztelés során ellenőrizzük, hogy a grafikus felület elemei helyesen működnek-e, és az aszinkron API hívások megfelelően frissítik-e a felhasználói felületet.

b. ***1.6.4.1.2. Teszterv***
   - **Tesztelési cél**: A szoftver üzleti folyamatainak (foglalás, szolgáltatáskezelés, adminisztráció) manuális ellenőrzése.
   - **Tesztelt modulok**: Bejelentkezés, Naptár nézet, Szolgáltatások szerkesztése, munkaidő beállítása, adminisztrációs panel.
   - **Tesztkörnyezet**:
     - **Operációs rendszer**: Windows 11 Pro, Android 11 (Emulátor)
     - **Környezet**: .NET 8 Desktop Runtime
   - **Tesztelési eszközök**: Manuális tesztelési sablon, Postman (API monitorozáshoz)

c. ***1.6.4.1.3. Tesztesetek***

Teszte set ID	Leírás	Lépések	Elvárt eredmény	Eredmény
TE001	Bejelentkezés érvényes adatokkal	1. Nyissa meg az appot. 2. írja be az admin emailt és jelszót 3. Kattintson a Bejelentkezés gombra	A rendszer belépteti a felhasználót, megjelenik a naptár vagy az admin panel.	Pass
TE002	Bejelentkezés hibás jelszóval	1. Adjon meg egy létező emailt 2. írjon be egy véletlenszerű jelszót 3. Kattintson a bejelentkezés gombra	Piros hibaüzenet jelenik meg ("Hibás jelszó"), a belépés sikertelen.	Pass
TE003	Új szolgáltatás létrehozása	1. Menjen a Szolgáltatásokhoz 2. Kattintson az Új (+) gombra. 3. Töltse ki a mezőket és mentse.	Az új szolgáltatás megjelenik a listában és az adatbázisban is.	Pass
TE004	Időpont "Kész"-nek jelölése	1. Kattintson egy "booked" státuszú kártyán a pipa ikonra.	A kártya színe zöldre változik, a státusza "completed" lesz.	Pass
TE005	Munkaidő mentése	1. A Munkaidő menüben állítson be új órákat. 2. Kattintson a Mentés gombra	Megjelenik a "Mentés kész!" felirat, az adatok frissülnek.	Pass
TE006	Felhasználó törlése (Admin)	1. Admin fiókkal menjen a kezelés panelre. 2. Kattintson egy vendég melletti Törlésre	A vendég eltűnik a listából, foglalásai lemondódnak.	Pass

1.6.4.2. Automata tesztelési dokumentáció

1.6.4.2.1. Cél meghatározása

Ez a dokumentáció a szoftverhez készült egységtesztek (Unit Tests) eredményeit foglalja össze.

1.6.4.2.2. Teszterv

- **Tesztelési cél**: Az üzleti logika és az adatformázás helyességének igazolása kód szinten.
- **Tesztelt modulok**: Model formázás (Service, User), Login UI állapotkezelés.
- **Tesztkörnyezet**: Windows 11 Pro, Visual Studio 2022.
- **Tesztelési eszközök**: MSTest framework, .NET Unit Testing Tools.

1.6.4.2.3. Automata tesztesetek

Automata Teszt ID	Leírás	Input Adatok	Elvárt eredmény	Eredmény
AT001	ServiceDisplayTextTest: Szolgáltatás szöveges megjelenítése	Name: "Hajvágás", Price: 5000, Duration: 30	"Hajvágás - 5000 Ft (30 perc)"	Pass
AT002	UserDisplayFormatTest: Felhasználó formázott neve	Name: "Teszt Elek", Email: "teszt@gmail.com"	"Teszt Elek (teszt@gmail.com)"	Pass
AT003	PasswordSVGTest: Kezdő jelszó ikon ellenőrzése	LoginViewModel inicializálása	Az ikon adatsora a zárt szem SVG-vel kezdődik.	Pass
AT004	PasswordRevealTest: Jelszó láthatóság váltása	TogglePasswordVisibility Command végrehajtása	Az IsPasswordVisible értéke TRUE-ra vált.	Pass

1.6.4.3. Teszt összegzés

Az MSTest segítségével futtatott automata tesztek igazolják, hogy az alkalmazás alapvető logikai egységei stabilak. A tesztek lefedik az adatmegjelenítési és a kritikus UI állapotkezelési funkciókat. A tesztek futtatása minden Build folyamatnál javasolt.

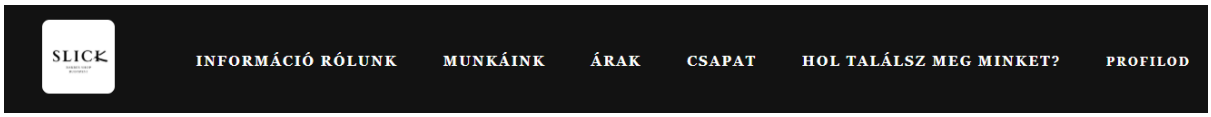
2. Felhasználói dokumentáció

2.1. Bevezetés

Ez a dokumentáció bemutatja a Slick Barber Shop rendszer használatát. A szoftver két fő részből áll: egy reszponzív (mobilbarát) **weboldalból**, ahol a vendégek böngészhetnek és időpontot foglalhatnak, valamint egy **asztali és mobil alkalmazásból**, amely a borbélyok és az üzletvezetők mindennapi munkáját segíti.

2.2. Vendégek számára (Webes felület)

A vendégek a böngészőből megnyitható weboldalon keresztül érhetik el a szalon szolgáltatásait.



1. ábra: Navigációs sáv

2.2.1. Navigáció

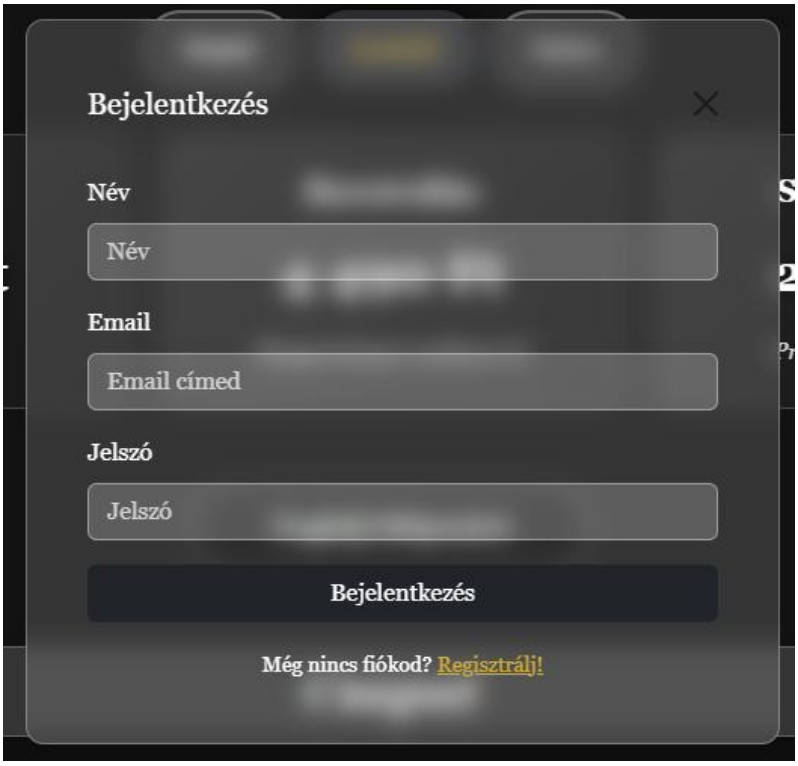
A főoldalon minden fontos információ megtalálható a szalonról. A felső navigációs sáv segítségével könnyedén eljuthat a következő részekhez:

- **Információ rólunk:** Rövid bemutatkozás az üzletről.
- **Munkáink:** Képgaléria a borbélyok korábbi munkáiról. Ha az egeret egy fekete-fehér képre húzza, az színessé válik.
- **Árak:** A szolgáltatások (Haj, Szakáll, Extra) árai kategóriákra bontva, füles (tab) elrendezésben tekinthetők meg.
- **Csapat:** Bemutatkozó kártyák a szalonban dolgozó fodrászokról.
- **Kapcsolat & Térkép:** Cím, e-mail, telefonszám és nyitvatartás. Itt található egy interaktív Google Térkép is az útvonaltervezéshez.

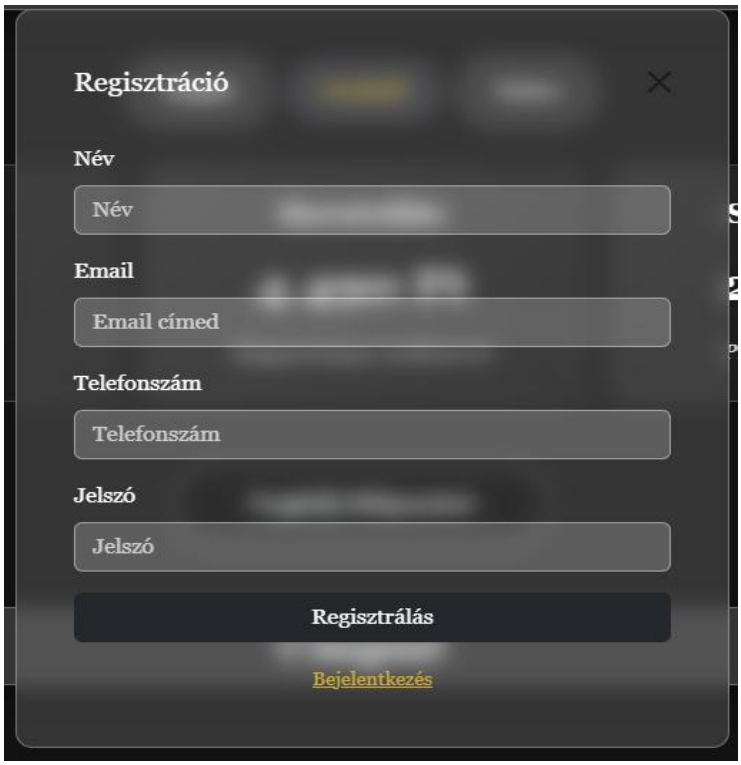
2.2.2. Regisztráció és bejelentkezés

Az időpontfoglaláshoz és a saját adatok kezeléséhez felhasználói fiókra van szükség.

1. Kattintson a fejlécben található **Profilod** vagy az oldalon lévő **Foglalj Időpontot** gombra.
2. A felugró ablakban válassza a **Regisztráció** vagy a **Bejelentkezés** lehetőséget.
3. Regisztrációhoz adja meg nevét, e-mail címét, jelszavát és telefonszámát.
4. Sikeres művelet esetén a rendszer egy felugró üzenetben tájékoztatja, majd be is lépteti.



2. ábra: Bejelentkezés ablak



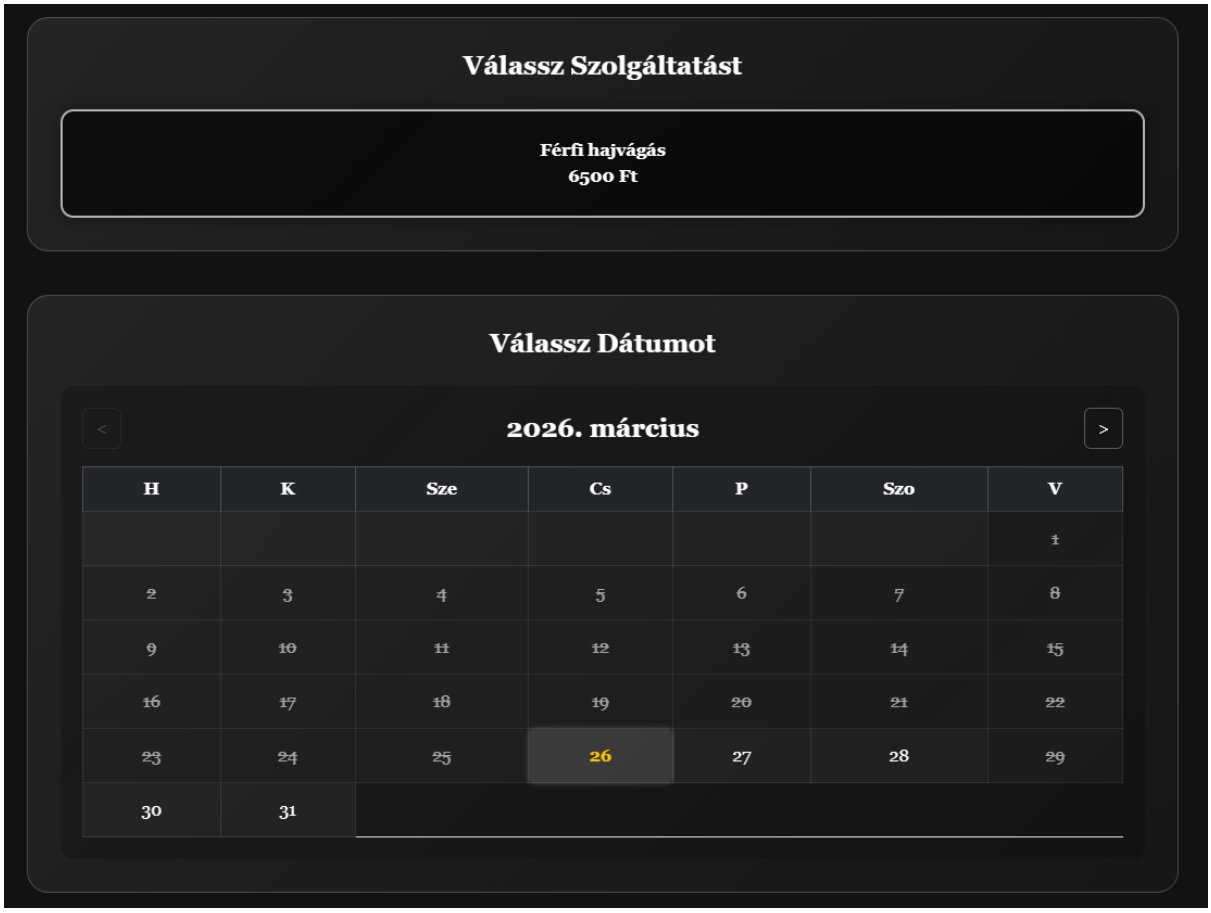
2. ábra: Regisztráció ablak

2.2.3. Időpontfoglalás menete

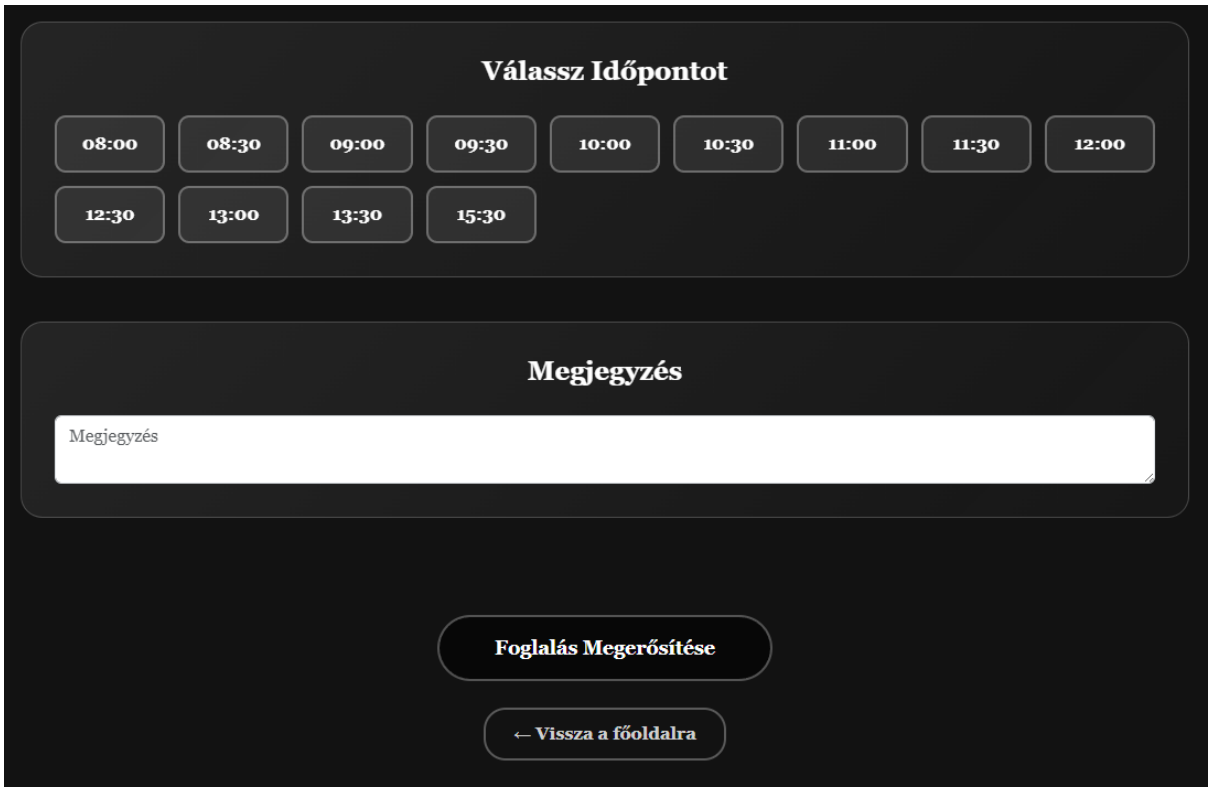
Az időpontfoglalás egy egyszerű, több lépésből álló folyamat:

1. **Fodrász kiválasztása:** Válassza ki a szimpatikus borbélyt a kártyákra kattintva.
2. **Szolgáltatás kiválasztása:** A rendszer megjeleníti az adott fodrászhoz tartozó szolgáltatásokat (pl. Hajvágás, Borotválás). Válasszon egyet.
3. **Dátum kiválasztása:** A naptárban válassza ki a kívánt napot (a múltbeli dátumok és a vasárnapok nem választhatók).
4. **Időpont kiválasztása:** Kattintson a szabad idősávok egyikére.
5. **Megjegyzés:** Opcionálisan üzenetet is hagyhat a fodrásznak.
6. **Véglegesítés:** Kattintson a *Foglalás Megerősítése* gombra.
   A sikeres foglalásról a rendszer automatikus megerősítő e-mailt küld.

3. ábra: Fodrász választás rész foglaláskor



4. ábra: Szolgáltatás és dátum választás foglaláskor

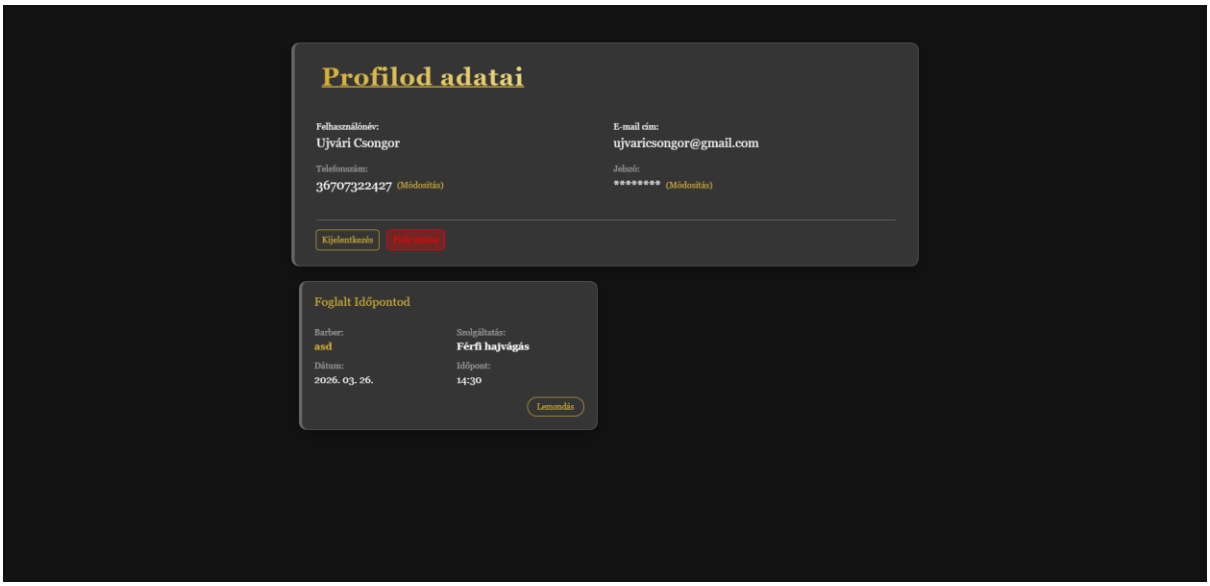


5. ábra: Időpont és megjegyzés foglaláskor

2.2.4. Profil kezelése és Foglalások lemondása

A bejelentkezett vendégek a **Profil** oldalon kezelhetik adataikat:

- **Adatok módosítása:** Itt megváltoztathatja telefonszámát vagy jelszavát. A mentés gombra kattintva az adatok frissülnek.
- **Foglalások megtekintése:** Az oldal alján kártyás formában láthatja az összes aktív foglalását (dátummal, időponttal, választott fodrásszal).
- **Időpont lemondása:** Bármely aktív foglalás kártyáján megtalálja a **Lemondás** gombot. A rendszer a véletlen kattintások elkerülése végett megerősítést kér.
- **Fiók törlése:** Lehetőség van a teljes felhasználói profil végleges törlésére is a piros *Fiók törlése* gombbal.
- **Kijelentkezés:** Ezzel a gombbal biztonságosan kiléphet a fiókjából.



5. ábra: A felhasználói profil oldal a foglalásokkal és lemondás gombbal

2.3. Borbélyok számára (Asztali és Mobil alkalmazás)

2.3.1. Telepítés és futtatás

A Slick Barber Manager alkalmazások használatához nem szükséges bonyolult telepítési folyamat, mivel mindkét platformra optimalizált, azonnal futtatható csomagokat készítettünk.

- **Asztali számítógép (Windows):**

Az asztali alkalmazás egy professzionális telepítőcsomagban (SlickBarberSetup.exe) érhető el, amely automatikusan elvégzi a szükséges beállításokat.

1. **Letöltés:** Látogasson el a projekt GitHub oldalára, és a **Releases** menüpont alatt töltse le a SlickBarber.v1.0.0.rar fájlt.
2. Kicsomagolás: Csomagolja ki a letöltött tömörített mappát.
3. **Indítás:** Indítsa el a letöltött telepítőfájlt.
4. **Telepítési folyamat:**
    a. Olvassa el és fogadja el a **Licencszerződést**.
    b. Válassza ki a telepítés helyét (alapértelmezetten a Program Files mappába kerül).
    c. A telepítés végén a rendszer elvégzi az automatikus konfigurációt.
5. **Windows SmartScreen:** Mivel az alkalmazás nem rendelkezik vásárolt digitális aláírással, a Windows figyelmeztetést dobhat. Ekkor kattintson a **„További információ”** linkre, majd a **„Futtatás mindenképpen”** gombra.
6. **Használat:** A telepítés után az alkalmazás ikonja megjelenik az **Asztalon** és a **Start menüben** is.

Rendszerkövetelmény: Windows 10 vagy újabb (64-bit). Külön .NET keretrendszer telepítése nem szükséges.

- **Mobiltelefon (Android):**
  A mobilalkalmazás egyetlen .apk telepítőcsomagban érhető el.

1. **Letöltés:** Töltse le a telefonjára a SlickBarberManager v1.1.1.apk fájlt a GitHub Release-ből vagy a weboldalon található linkről.
2. **Engedélyezés:** Amennyiben a telefon még nem engedélyezi, a telepítés megkezdésekor felugró ablakban válassza a beállításokat, és kapcsolja be az **„Ismeretlen forrásokból származó alkalmazások telepítése”** opciót (vagy engedélyezze a böngészőnek a telepítést).
3. **Telepítés:** Nyissa meg a letöltött fájlt, és nyomjon a **„Telepítés”** gombra.
4. **Google Play Protect:** A Google figyelmeztetheti, hogy a fejlesztő ismeretlen. Kattintson a lefelé mutató kis nyílra, majd a **„Telepítés mindenképpen”** lehetőségre.

2.3.2. Belépés

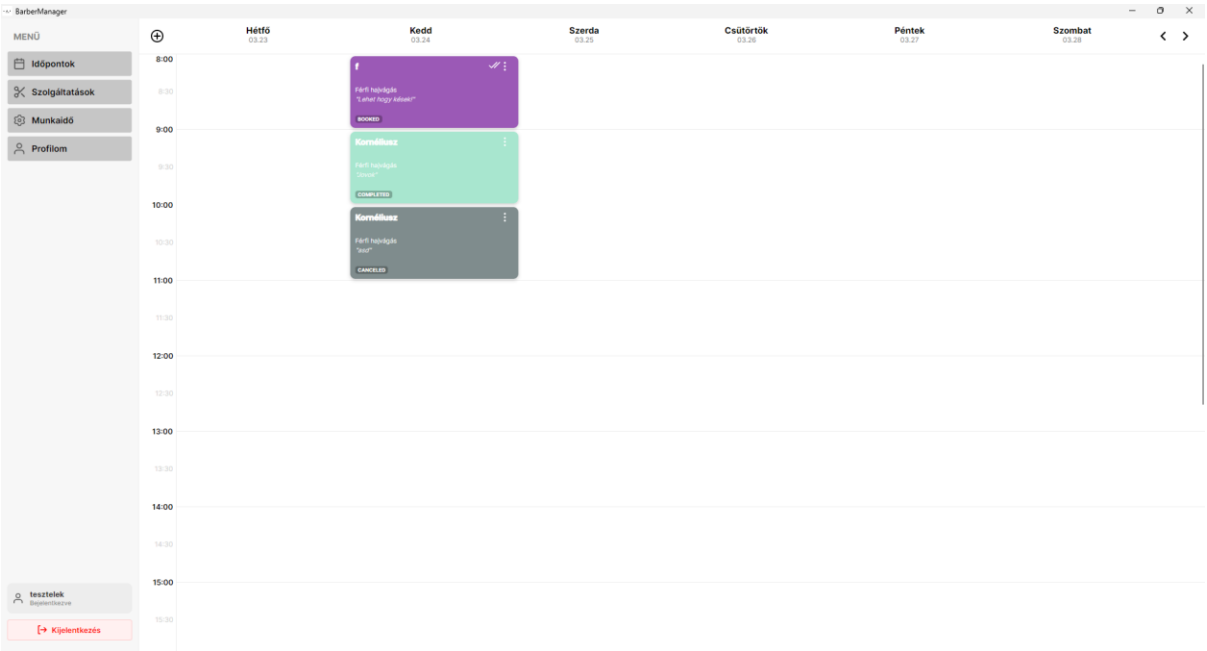
Az alkalmazás elindítása után az e-mail cím és a jelszó megadásával léphet be. A jelszómező melletti "szem" ikonra kattintva a jelszó láthatóvá tehető.

1. ábra: Bejelentkezési képernyő

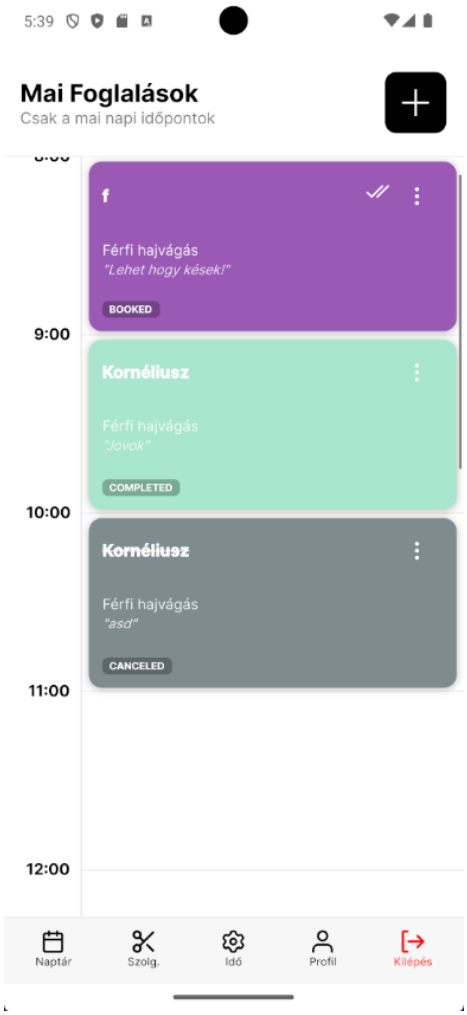
2.3.3. Naptár és Időpontok kezelése

Belépés után a főképernyő a naptárat mutatja. Asztali gépen **heti naptárnézet**, míg mobilon a jobb olvashatóság érdekében **napi, görgethető listás nézet** fogadja.

- **Új foglalás felvétele (Kézi):** Kattintson az "Új foglalás" (+) gombra, válasszon egy regisztrált vendéget, szolgáltatást, napot és idősávot.
- **Foglalás részletei:** Egy meglévő kártyán a három pontra (Részletek) kattintva láthatja a vendég nevét, telefonszámát, és a foglaláshoz fűzött megjegyzését.
- **Teljesítés:** Ha a vendég kiszolgálása befejeződött, kattintson a **"pipa"** gombra. A kártya színe zöldre (Completed) változik.
- **Lemondás:** A részletek között található *Lemondás* gombbal törölheti az időpontot. Ilyenkor a kártya szürke lesz, a vendég neve pedig áthúzva jelenik meg (Canceled).



2. ábra: Asztali naptár nézet a foglalásokkal

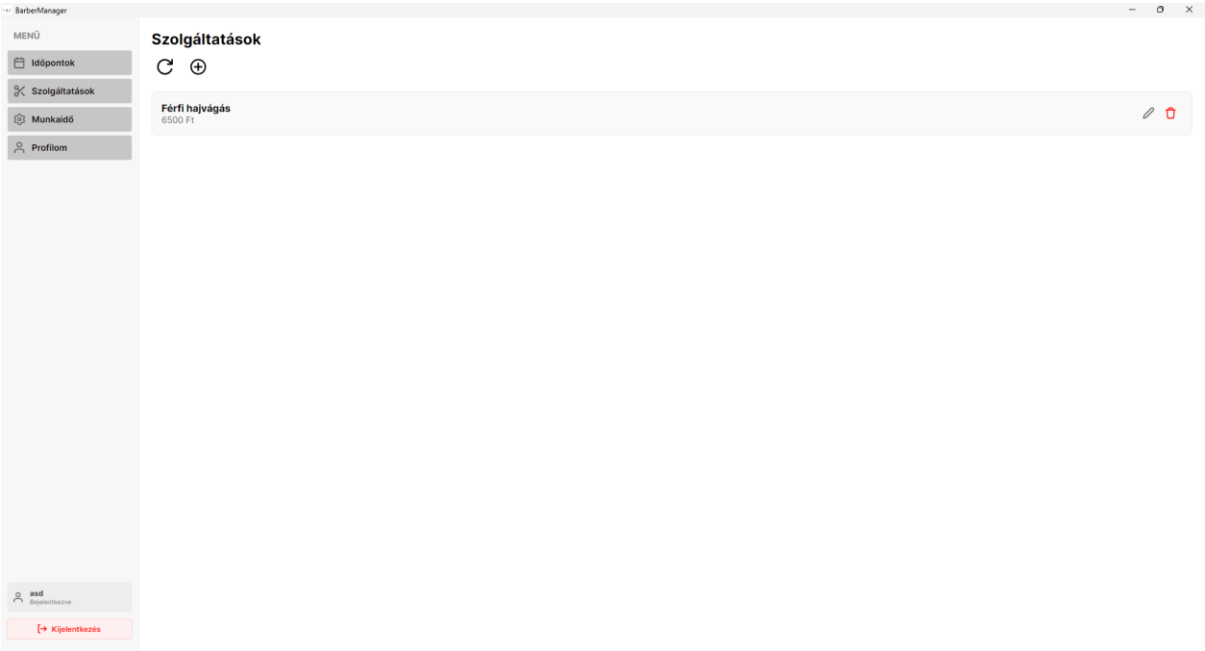


4. ábra: Napi listás nézet mobil eszközön

2.3.4. Szolgáltatások és Munkaidő beállítása

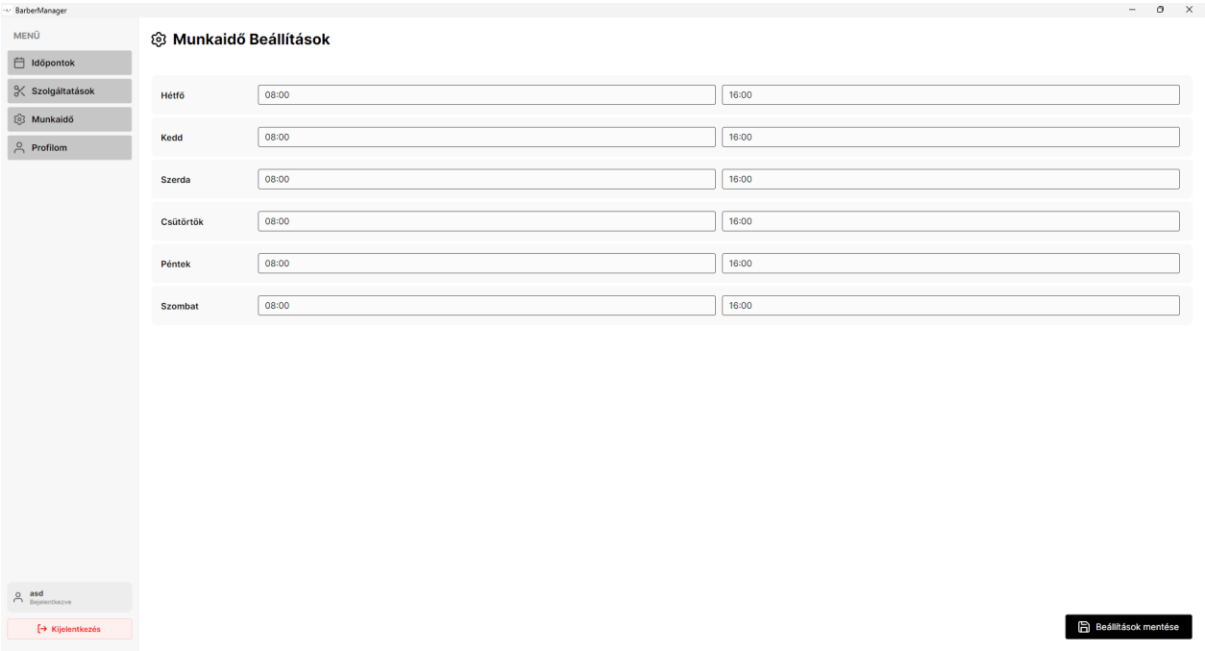
A bal oldali (mobilon alsó) menüsor segítségével személyre szabhatja fiókját:

- **Szolgáltatások:** Listázza a saját szolgáltatásait. Az **Új hozzáadása (+)** gombbal felvehet új hajvágást/szakállvágást, megadva annak nevét, leírását, árát és időtartamát. A meglévő kártyákon a ceruza ikonnal szerkeszthet, a kuka ikonnal törölhet.



5. ábra: Szolgáltatások lista nézet

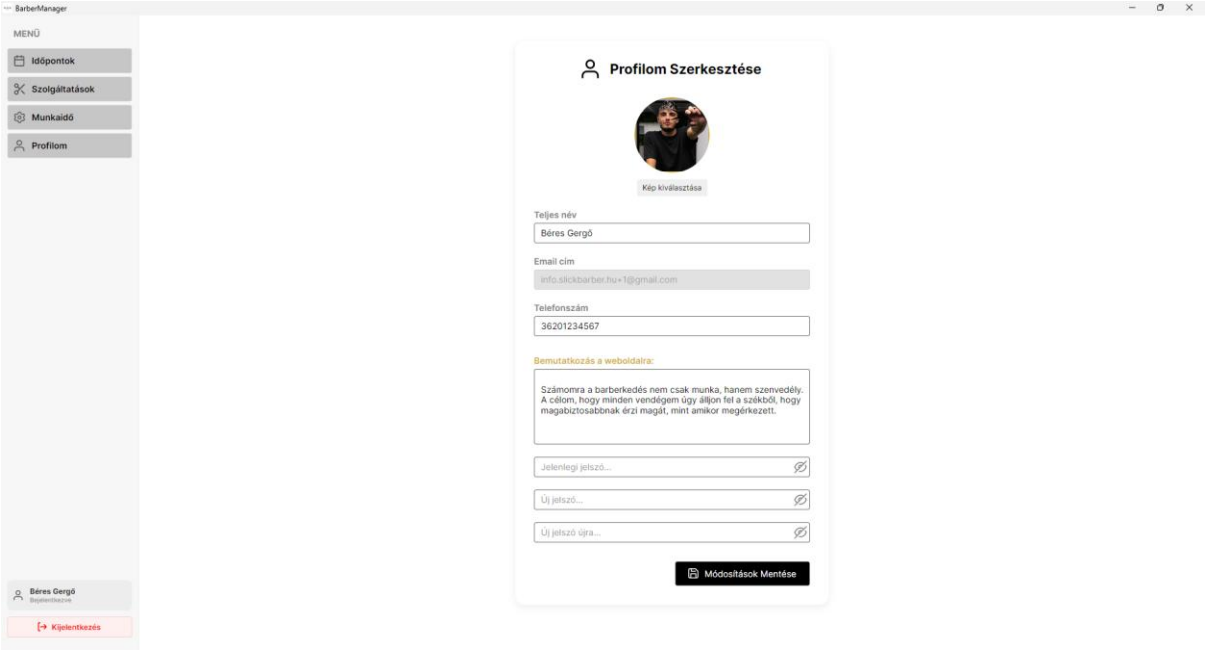
- **Munkaidő:** A hét napjaira (Hétfőtől Vasárnapig) lebontva beállíthatja, hogy mettől meddig dolgozik. A módosításokat a *Beállítások mentése* gombbal rögzítheti. A vendégek a weboldalon csak a megadott intervallumon belül tudnak foglalni.



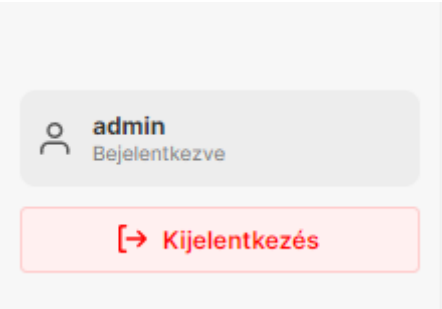
6. ábra: Munkaidő beállítások nézet

2.3.5. Profil és kijelentkezés

A "Profilom" menüpontban a fodrászok teljeskörűen személyre szabhatják fiókjukat. Lehetőség van a név, telefonszám és jelszó módosítására, valamint **egy profilkép feltöltésére és egy rövid bemutatkozó szöveg megadására**, amely a vendégek számára a weboldalon (a Csapat szekcióban) is megjelenik. Mobil eszközön a felület görgethető, a mentés gomb az oldal alján található. A menüsor alján lévő piros Kijelentkezés gombbal léphet ki az alkalmazásból.



7. ábra: Profil menüpont



8. ábra: Kijelentkezés gomb & bejelentkezett fodrász

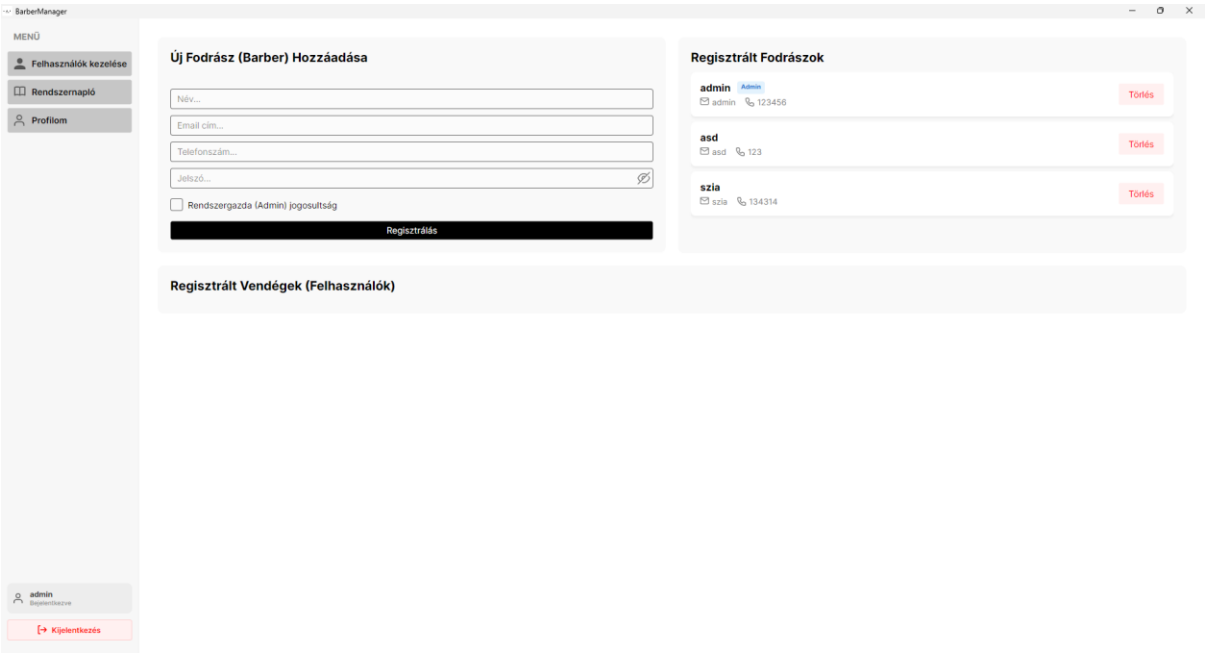
2.4. Adminisztrátorok számára (Üzletvezetők)

Az "Admin" jogosultsággal rendelkező fodrászok az alkalmazásban plusz két menüpontot is látnak a normál borbélyokhoz képest. Ezek a bal oldali menüből érhetők el.

2.4.1. Felhasználók kezelése (Admin Panel)

Ezen a felületen kezelheti a rendszerben lévő dolgozókat és vendégeket.

- **Új fodrász (Barber) hozzáadása:** Töltse ki a nevet, e-mailt, telefont és jelszót. Ha az új dolgozónak is szeretne vezetői jogokat adni, pipálja be az *"Admin jogosultság"* jelölőnégyzetet, majd kattintson a Regisztrálás gombra.
- **Fodrászok és Vendégek törlése:** Az oldalon lejjebb görgetve láthatja a regisztrált fodrászok és vendégek listáját. Egy felhasználó melletti **Törlés** gombra kattintva véglegesen eltávolíthatja őt a rendszerből. *(Figyelem: Vendég törlése esetén a rendszer automatikusan lemondja a vendég összes jövőbeli foglalását is!)*

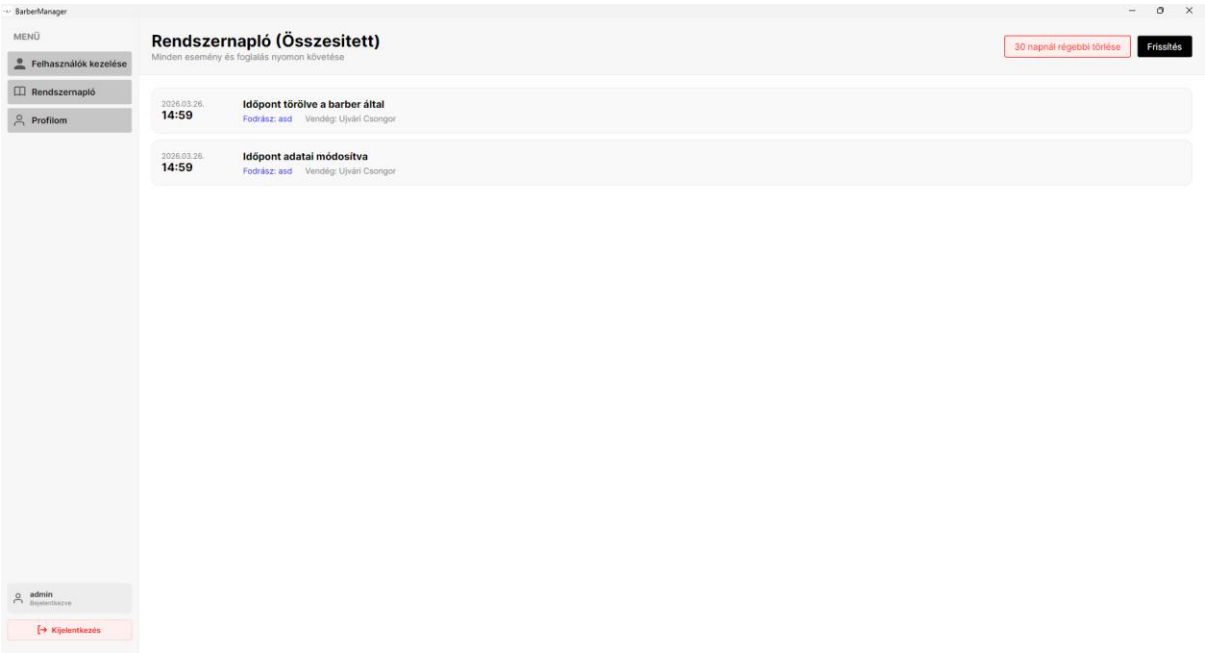


9. ábra: Adminisztrációs felület a felhasználók kezeléséhez

2.4.2. Rendszernapló (Logs)

A napló (Logs) menüpontban nyomon követheti a szalon életét.

- **Megtekintés:** A felület időrendben listázza az összes rendszerszintű eseményt (ki, mikor, mit módosított, milyen foglalást adott le vagy mondott le).
- **Takarítás:** A napló túlcsordulásának elkerülése érdekében a **"30 napnál régebbi törlése"** gombbal egyetlen kattintással eltávolíthatja az elévült naplóbejegyzéseket a rendszerből.



10. ábra: Adminisztrációs felület a rendszernaplókhoz